

算法设计与分析

概述

算法

概念

- 算法是指解题方案的准确而完整的描述，是一系列解决问题的清晰指令，算法代表着用系统的方法描述解决问题的策略机制。
- 能够对一定规范的输入，在有限时间内获得所要求的输出。
- 如果一个算法有缺陷，或不适合于某个问题，执行这个算法将不会解决这个问题。
- 不同的算法可能用不同的时间、空间或效率来完成同样的任务。
- 一个算法的优劣可以用**空间复杂度**与**时间复杂度**来衡量。

性质

算法是若干指令的有穷序列，满足性质：

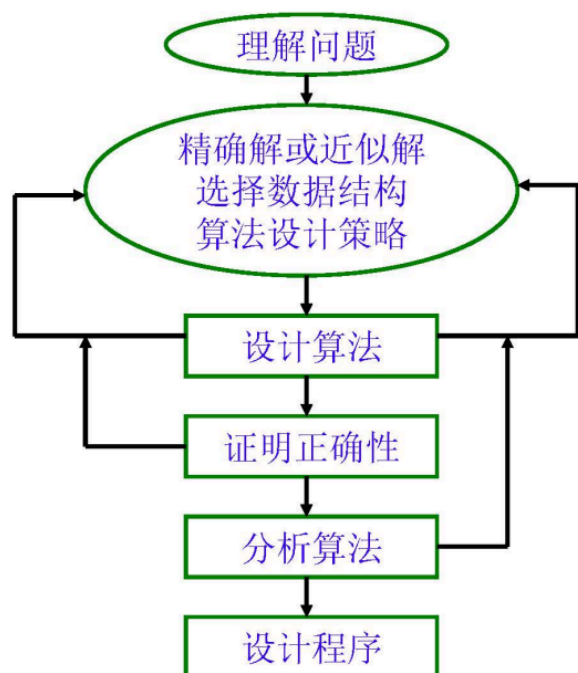
- 输入：有零或若干个外部提供的量作为算法的输入
- 输出：算法产生至少一个量作为输出
- 确定性：组成算法的每条指令是清晰、无歧义的
- 有限性：算法中每条指令的执行次数是有限的，执行每条指令的时间也是有限的

程序

概念与性质

- 程序是算法用某种程序设计语言的具体实现
- 程序可以不满足算法的性质 4（有限性）
 - *e.g.* 操作系统：一个在无限循环中执行的程序，因此不是一个算法
 - 操作系统的各种任务可看成是单独的问题，每一个问题由操作系统中的一个子程序通过特定的算法来实现。该子程序得到输出结果后便终止。

问题求解



算法复杂性

算法复杂性

- 算法复杂性 = 算法所需的计算机资源
- 分类 (n 是问题的规模, 即输入大小):
 - 时间复杂度 $T(n)$
 - 空间复杂度 $S(n)$
- $C = F(N, I, A)$
 - 变量:
 - C : 算法复杂性
 - N : 解的问题的规模
 - I : 算法输入
 - A : 算法本身
 - $F(N, I, A)$: 由参数确定的三元函数, 时间为 τ , 空间为 s
 - 通常 A 隐含在复杂性函数名当中, 简写为 $T = T(N, I) / S = S(N, I)$

时间复杂性

- 最坏情况下: $T_{max}(n) = \max\{T(I) | \text{size}(I) = n\}$
- 最好情况下: $T_{min}(n) = \min\{T(I) | \text{size}(I) = n\}$
- 平均情况下: $T_{avg}(n) = \sum_{\text{size}(I)=n} p(I)T(I)$, 其中 I 是问题的规模为 n 的实例, $p(I)$ 是实例 I 出现的概率。
- 主定理:
 - Case1: $n^{\log_b a} > f(n)$, $O(n^{\log_b a})$
 - Case2: $n^{\log_b a} = f(n)$, $O(n^{\log_b a} \cdot \log n)$

- Case3: $n^{\log_b a} < f(n)$, $O(f(n))$

算法渐进复杂性

- 概念
 - As $n \rightarrow \infty$, $T(n) \rightarrow$
 - As $n \rightarrow \infty$, $(T(n) - t(n))/T(n) \rightarrow 0$
 - $t(n)$ 是 $T(n)$ 的渐进性态, 为算法的渐进复杂度。
 - 数学上, $t(n)$ 是 $T(n)$ 的渐进表达式, 是其去掉低阶项留下的主项。
 - 换句话说, n 趋近于无穷时, $t(n)$ 渐进于 $T(n)$, 可以替代其作为算法的复杂性度量。
- 渐进分析记号 ($\forall n, f(n), g(n) \geq 0$)
 - 渐进上界符号 O : $f(n) = O(g(n))$ 代表 c 和 n_0 , 使得对所有 $n \geq n_0$ 有: $0 \leq f(n) \leq c \cdot g(n)$
 - 非紧上界符号 o : $f(n) = o(g(n))$ 代表 c 和 n_0 , 使得对所有 $n \geq n_0$ 有: $0 \leq f(n) < c \cdot g(n)$
 - 渐进下界符号: $f(n) = \Omega(g(n))$ 代表 c 和 n_0 , 使得对所有 $n \geq n_0$ 有: $0 \leq c \cdot g(n) \leq f(n)$
 - 渐进上界符号: $f(n) = \Theta(g(n))$ 代表 c 和 n_0 , 使得对所有 $n \geq n_0$ 有: $0 \leq c \cdot g(n) < f(n)$
 - $f(n) \in \Theta(g(n))$ $g(n) \in \Theta(f(n))$
 - 紧渐进界记号: $f(n) = \Theta(g(n))$, 代表 c_1, c_2 和 n_0 , 使得对所有 $n \geq n_0$ 有: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$
- 定理:
 - $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$
 - $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$ (定义可证)
 - $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
 - $O(f(n)) * O(g(n)) = O(f(n) * g(n))$

递归和分治策略

整体思想:

- 将一个难以直接解决的大问题, 分割成一些规模较小的相同问题, 以便各个击破, 分而治之。(分治)
- 对 k 个子问题分别求解。如果子问题的规模仍然不够小, 则再划分为 k 个子问题, 如此递归的进行下去, 直到问题规模足够小, 很容易求出其解为止。
- 将求出的小规模的问题的解合并为一个更大规模的问题的解, 自底向上逐步求出原来问题的解。

递归

递归概念

- 递归算法: 直接或间接地调用自身的算法
- 递归函数: 用函数自身给出定义的函数

- 两个要素：**边界条件**和**递归方程** =>有限次
- 由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的发生。

实例

阶乘

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

Fibonacci数列

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

Ackerman函数（双递归函数）

$$A(n, m) = \begin{cases} A(1, 0) = 2 & \\ A(0, m) = 1 & m \geq 0 \\ A(n, 0) = n + 2 & n \geq 2 \\ A(n, m) = A(A(n-1, m), m-1) & n, m \geq 1 \end{cases}$$

- 双递归函数：一个函数及它的一个变量是由函数自身定义。
- 求解：
 - $M = 0$: $A(n, 0) = n + 2$
 - $M = 1$: 首先计算可以得到 $A(1, 1) = A(A(0, 1), 0) = A(1, 0) = 2$, 且 $A(n, 1) = A(A(n-1, m), 0) = A(n-1, 1) + 2$, 递推可得 $A(n, 1) = 2 * n$
 - $M = 2$: $A(n, 2) = A(A(n-1, 2), 1) = 2A(n-1, 2)$, 且 $A(1, 2) = A(A(0, 2), 1) = A(1, 1) = 2$, 递推可得 $A(n, 2) = 2^n$;
 - $M = 3$: 可以推出为 $2^{2^{2^{\dots^2}}}$
 - M 更大无法表示
- => 无法找到非递归的定义

排列问题

- 问题描述：设计一个递归算法生成n个 $\{r_1, \dots, r_n\}$ 元素的全排列。设 $R = \{r_1, r_2, \dots\}$ 是要进行排列的n个元素， $R_i = R - \{r_i\}$ ，集合X中元素的全排列记为 $perm(X)$ 。 $(r_i)perm(X)$ 表示在全排列 $perm(X)$ 的每个排列前加上前缀得到的排列。
- 归纳：

$$perm(R) = \begin{cases} (r) & n = 1, r \text{ 是 } R \text{ 中唯一的元素} \\ (r_1)perm(R_1) \cup \dots \cup (r_n)perm(R_n) & n > 1 \end{cases}$$

- 实现：

```

template <class Type>
void Perm (Type list[], int k, int m) { // 产生list[k:m]的所有排列
    if (k == m) { // 只剩下一个元素，到达递归边界
        for (int i = 0; i <= m; i++)
            cout << list[i] << " ";
        cout << endl;
    }
    else { // 还有多个元素待排列，递归产生排列
        for (int i = k; i <= m; i++) {
            Swap(list[k], list[i]); // 把第 i 个元素交换到当前位置 k
            Perm(list, k + 1, m); // 递归处理 k+1 及其后面的元素
            Swap(list[k], list[i]); // 回溯：恢复原来的顺序，以便进行下一次循环
        }
    }
}

```

整数划分问题：

- 问题描述：将正整数 n 表示成一系列正整数之和，求不同划分个数
- 解决思路：增加变量 m 代表当前最大的加数不大于 m ，记录划分数为 $q(n, m)$ ，递归关系建立为：

$$q(n, m) = \begin{cases} 1 & n = 1 \text{ 或 } m = 1 \\ q(n, n) & m > n \\ 1 + q(n, n - 1) & n = m \\ q(n, m - 1) + q(n - m, m) & n > m > 1 \end{cases}$$

- 解释（按顺序）：
 - 全分解为1，唯一情况
 - 显然 m 大时，最大的加数只能是 n
 - 分两种情况，一种是分解成自己（ $n=n$ ），另一种是最大加数变成 $n-1$
 - 最常规情况：一种是没有 m ，最大加数变成 $m-1$ ；另一种是最大加数等于 m ，去掉一个 m 后继续分解
- 实现：

```

int q(int n, int m) {
    if (n < 1 || m < 1) return 0;

    if (n == 1 || m == 1) return 1;

    if (m > n) return q(n, n);

    if (m == n) return 1 + q(n, m - 1);

    if (n > m && m > 1) return q(n, m - 1) + q(n - m, m);
}

```

```
return 0;
}
```

Hanoi塔：

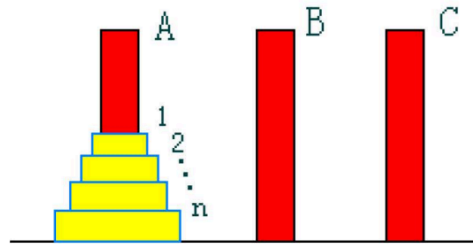
- 问题描述：

设a,b,c是3个塔座。开始时，在塔座a上有一叠共n个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为1,2,...,n,现要求将塔座a上的这一叠圆盘移到塔座b上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

规则1：每次只能移动1个圆盘；

规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；

规则3：在满足移动规则1和2的前提下，可将圆盘移至a,b,c中任一塔座上。



- 解决思想：将1到n-1个圆盘作为**整体**，通过c柱从a柱移动到b柱，然后将第n个圆盘移动到c柱，最后将1到n-1圆盘通过a柱移动到c柱
- 实现：

```
void Hanoi(int n, int a, int b, int c) {
    if(n == 1) {
        move(n, a, c);
        return;
    }

    Hanoi(n - 1, a, c, b);
    move(n, a, c);
    Hanoi(n - 1, b, a, c);
}
```

求解线性递归关系

1. 特征方程，求解特征根 r_i
2. 根据特征根列出通解： $x_n = \sum c_i \cdot r_i^n$
3. 代入初值条件，求解 c_i
4. 代入，获得答案

• e.g.

(2) 求解线性递归关系:
$$\begin{cases} x_{n+1} = x_n + 12x_{n-1}, n \geq 1 \\ x_0 = 1, x_1 = 0.5 \end{cases}$$

解: 该关系的特征方程

$$r^2 = r + 12$$

其特征根为

$$r_1 = 4, r_2 = -3$$

故通解为

$$x_n = c_1 \cdot 4^n + c_2(-3)^n$$

由初值条件

$$x_0 = 1, x_1 = 0.5$$

得

$$\begin{cases} 1 = c_1 + c_2 \\ 0.5 = 4c_1 - 3c_2 \end{cases}$$

求得

$$c_1 = 0.5, c_2 = 0.5$$

故

$$x_n = \frac{1}{2} (4^n + (-3)^n)$$

小结

优点 => 方便

- 结构清晰, 可读性强
- 容易用数学归纳法证明算法正确性

缺点 => 效率低

递归算法的运行效率较低, 无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

解决方法

在递归算法中消除递归调用, 使其转化为非递归算法, 具体有几个途径:

- 采用用户定义的栈模拟系统的递归调用工作栈。该方法通用性强, 但是本质上还是递归, 由人工代替编译器的部分工作, 优化效果不明显。
- 用递推实现递归函数 (DP)。
- 通过变换, 将一些递归转换成尾递归, 从而迭代求出结果。

分治

思想

适用条件/特征

- 该问题的规模缩小到一定的程度就可以容易地解决
- 该问题可以分解为若干个规模较小的相同问题，即问题具有最优子结构性质
- 利用该问题分解出的子问题的解可以合并为该问题的解
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题（重要）

基本步骤

- 解决小规模问题（给定条件，提供方法）
- 分解问题
- 递归的解各个子问题
- 将各个子问题的解合并成原问题的解

复杂性分析

- 初始计算：

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

- 解释：
 - n ：问题的总规模。
 - k ：把问题分成了 k 个子问题。
 - n/m ：每个子问题的规模是原来的 $1/m$
 - $f(n)$ ：分解+合成的代价
 - $T(n)$ ：解决规模为 n 的问题所需的总时间

- 通解：

$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j \cdot f\left(\frac{n}{m^j}\right)$$

- 解释：
 - 左半部分：叶子结点的开销，根据子问题规模推导出递归树的高度为 $\log_m n$ ，且第 i 层有 k^i 个节点，则共有 $k^{\log_m n}$ 个叶子结点，换底公式计算后即为左半部分；
 - 右半部分：内部节点开销（分解合成），每层有 k^j 个节点，每个节点的分解合成开销为 $f(\frac{n}{m^j})$ ，最终计算结果为右半部分。

说明

- 用分治法时，最好使子问题的规模大致相同，这种做法出自平衡子问题的思想，如此效率几乎总是最好的
- 递归方程及其解只给出 n 等于 m 的方幂时 $T(n)$ 的值，但如果认为 $T(n)$ 足够平滑，那么由 n 等于 m 的方幂时 $T(n)$ 的值可以估计 $T(n)$ 的增长速度。通常假定 $T(n)$ 是单调上升的，从而：

$$m^i \leq n < m^{i+1} \text{ 时, } T(m^i) \leq T(n) < T(m^{i+1})$$

实例

(重要) 二分搜索

- 问题描述：给定有序（升序）的n个元素 $a[0:n-1]$ ，找出其中的某个特定元素 x
- 分析：符合分治特征，尤其是符合独立特征
- 实现（也可以递归实现，在通过大小比较确定的分区递归调用，此时参数需要传入 l 和 r ）：

```
int BinarySearch(const vector<int>& a, int x) {
    int l = 0;
    int r = a.size() - 1;

    // 必须有等于，因为相等时依然有一个元素需要比较
    while (r >= l) {
        // left + (right - left) / 2 可以防止溢出
        middle = (l + r) / 2;
        if (x == a[middle]) return middle;
        if (x < a[middle]) r = middle - 1;
        else l = middle + 1;
    }

    // 没找到
    return -1;
}
```

大整数乘法

- 问题描述：进行两个n位大整数的乘法运算
- 思想：
 - 将 n 位大整数平均分解为 $n/2$ 的高位和低位，然后利用分配率，可以防止计算机的溢出等物理问题
 - 拆分： $X = a \cdot 2^{n/2} + b$ ，由于底层为二进制，此处根据二进制进行高低位拆分，分成 a 和 b 两部分
 - 当简单的仅仅拆分成两部分分配相乘时，**计算效率并未提高**
 - 计算： $X \cdot Y = (a \cdot 2^{n/2} + b) \cdot (c \cdot 2^{n/2} + d) = ac \cdot 2^n + (ad + bc) \cdot 2^{n/2} + bd$
 - 计算过程中，将一次n位整数的乘法转换成了4次n/2位整数的乘法，即 $k=4, m=2$
 - 时间复杂度： $T(n) = 4T(n/2) + O(n)$ ，解得 $T(n) = O(n^2)$
 - 左：拆分后的子问题的解决时间
 - 右：拆分合并过程消耗，实际上拆分就是移位操作，合并就是加法过程
 - 移位操作（拆分）：左移实际上是数组拷贝过程，开辟一个新的内存空间，将一半原来的数据拷贝到前面，然后补0，复杂度 $O(n)$

- 加法操作（合并）：本质上是遍历以数组存储的大整数每一位，复杂度 $O(n)$
- 综上， $f(n)$ 的值为 $O(n)$
- 改进：本质上就是将乘法转换成加法
 - 核心思想：通过利用 ac 和 bd 的计算结果，将中间的两次乘法（ ad 和 bc ）转换成一次 $((a+b)(c+d))$
 - 公式： $ad + bc = (a+b)(c+d) - ac - bd$
 - 复杂度：
 - 将一次 n 位整数的乘法转换成了3次 $n/2$ 位整数的乘法，即 $k = 3, m = 2$
 - 递归方程： $T(n) = 3T(n/2) + O(n)$ ，解得 $T(n) = O(n^{\log_2 3}) = O(n^{1.5})$
 - 左：减少一次乘法
 - 右：增加的加法依然是线性增加的，所以依然是 $O(n)$
- 实现（辅助函数省略）：

```
#include <iostream>
#include <string>
#include <algorithm>
#include <vector>

using namespace std;

string multiply(string num1, string num2) {
    // 保证 num1 > num2, 输入为01串
    // pad: 对其长度
    // add/sub: 大数加/减法

    // 预处理：对齐长度
    pad(num1, num2);
    int n = num1.size();

    // 分割：将 num 分为高位和低位
    //  $X = a * 2^m + b$ 
    int m = n / 2;

    string a = num1.substr(0, n - m);
    string b = num1.substr(n - m);
    string c = num2.substr(0, n - m);
    string d = num2.substr(n - m);

    // 递归计算三次乘法
    //  $z0 = a * c$ 
    string z0 = multiply(a, c);

    //  $z1 = b * d$ 
    string z1 = multiply(b, d);
```

```

// z2 = (a + b) * (c + d)
string sum_ab = add(a, b);
string sum_cd = add(c, d);
string z2 = multiply(sum_ab, sum_cd);

// 计算中间项
// middle = ad + bc = z2 - z0 - z1 = (a+b)(c+d) - ac - bd
string middle = sub(sub(z2, z0), z1);

// 合并
// res = z0 * 2^(2m) + middle * 2^m + z1
string term1 = leftShift(z0, 2 * m); // z0 左移 2m 位
string term2 = leftShift(middle, m); // middle 左移 m 位
string res = add(add(term1, term2), z1);

return res;
}

```

Strassen矩阵乘法

- 思路与大整数乘法相同：将乘法转换成加法
- 问题描述：计算 $N \times N$ 的结果 ($C = A \times B$)
- 传统方法： $O(n^3)$
- 传统分治：将矩阵切分成4个 $N/2 \times N/2$ 的子矩阵块：

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

- 此时需要计算4个子块：

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

- 复杂度： $T(n) = 8T(n/2) + O(n^2)$
 - 左：显然从1次乘法变成8次，同时，矩阵乘法的规模指的是矩阵的维度，所以规模变成了一半
 - 右：矩阵加法的复杂度体现在遍历二维数组，即 $O(n^2)$
- 改进：Strassen 算法（两个 2×2 矩阵的乘积使用7次乘法是必须的）
 - 将8次乘法改进为7次，构造了7个中间矩阵 $M_1 - M_7$ 推导出 $C_1 - C_4$

$$\begin{aligned} M_1 &= A_{11}(B_{12} - B_{22}) \\ M_2 &= (A_{11} + A_{12})B_{22} \\ M_3 &= (A_{21} + A_{22})B_{11} \\ M_4 &= A_{22}(B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ M_6 &= (A_{12} - A_{22})(B_{21} + B_{22}) \\ M_7 &= (A_{11} - A_{21})(B_{11} + B_{12}) \end{aligned}$$

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

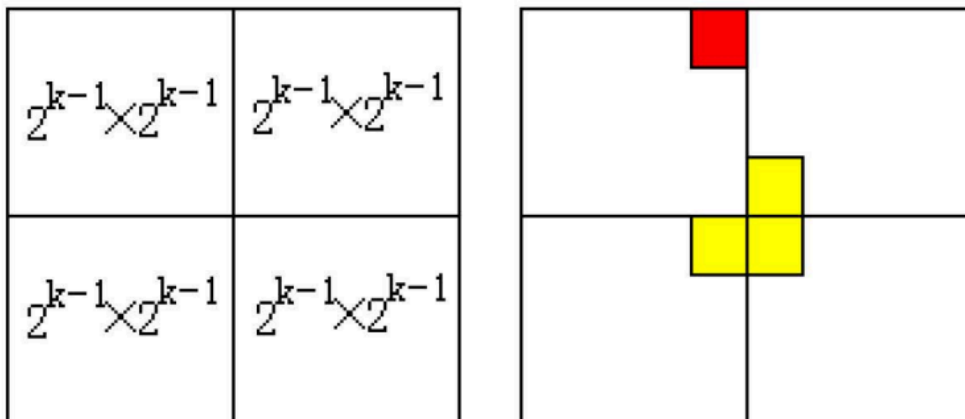
$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$

- 复杂度： $T(n) = 7T(n/2) + O(n^2) \Rightarrow T(n) = O(n^{\log_2 7}) = O(n^{2.81})$
- 实现：很复杂，略过了

棋盘覆盖

- 问题描述：在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。
- 处理思路：
 - 当 $k > 0$ 时，将 $2^k \times 2^k$ 棋盘分割为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘。特殊方格必位于4个较小子棋盘之一中，其余3个子棋盘中无特殊方格。
 - 为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，从而将原问题转化为4个较小规模的棋盘覆盖问题。
 - 递归地使用这种分割，直至棋盘简化为棋盘 1×1 。



- 实现：

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

// 定义全局变量
int tile = 1; // 骨牌编号，从 1 开始
const int MAX_SIZE = 1024; // 假设最大棋盘大小
int board[MAX_SIZE][MAX_SIZE]; // 棋盘数组

/**
 * 棋盘覆盖分治算法
 * @param tr 当前棋盘左上角的行号 (Top Row)
 * @param tc 当前棋盘左上角的列号 (Top Column)
 * @param dr 特殊方格的行号 (Distinguished Row)
 */
```

```

* @param dc    特殊方格的列号 (Distinguished Column)
* @param size  当前棋盘的边长 (必须是 2 的 k 次幂)
*/
void chessBoard(int tr, int tc, int dr, int dc, int size) {
    // 1. 递归边界
    if (size == 1) return;

    int t = tile++;          // 当前 L 型骨牌的编号
    int s = size / 2;        // 分割棋盘, s 为子棋盘的大小

    // --- 2. 处理左上角子棋盘 ---
    if (dr < tr + s && dc < tc + s) {
        // 特殊方格在此子棋盘中
        chessBoard(tr, tc, dr, dc, s);
    } else {
        // 特殊方格不在此子棋盘中
        // 用 t 号 L 型骨牌覆盖该子棋盘的右下角
        board[tr + s - 1][tc + s - 1] = t;
        // 将被覆盖的方格视为新的特殊方格, 递归处理
        chessBoard(tr, tc, tr + s - 1, tc + s - 1, s);
    }

    // --- 3. 处理右上角子棋盘 ---
    if (dr < tr + s && dc >= tc + s) {
        // 特殊方格在此子棋盘中
        chessBoard(tr, tc + s, dr, dc, s);
    } else {
        // 特殊方格不在此子棋盘中
        // 用 t 号 L 型骨牌覆盖该子棋盘的左下角
        board[tr + s - 1][tc + s] = t;
        // 将被覆盖的方格视为新的特殊方格, 递归处理
        chessBoard(tr, tc + s, tr + s - 1, tc + s, s);
    }

    // --- 4. 处理左下角子棋盘 ---
    if (dr >= tr + s && dc < tc + s) {
        // 特殊方格在此子棋盘中
        chessBoard(tr + s, tc, dr, dc, s);
    } else {
        // 特殊方格不在此子棋盘中
        // 用 t 号 L 型骨牌覆盖该子棋盘的右上角
        board[tr + s][tc + s - 1] = t;
        // 将被覆盖的方格视为新的特殊方格, 递归处理
        chessBoard(tr + s, tc, tr + s, tc + s - 1, s);
    }

    // --- 5. 处理右下角子棋盘 ---
    if (dr >= tr + s && dc >= tc + s) {
        // 特殊方格在此子棋盘中
        chessBoard(tr + s, tc + s, dr, dc, s);
    }
}

```

```

    } else {
        // 特殊方格不在此子棋盘中
        // 用 t 号 L 型骨牌覆盖该子棋盘的左上角
        board[tr + s][tc + s] = t;
        // 将被覆盖的方格视为新的特殊方格，递归处理
        chessBoard(tr + s, tc + s, tr + s, tc + s, s);
    }
}

// 测试用的主函数
int main() {
    int size = 4; // 棋盘大小，必须是 2 的幂 (2, 4, 8, 16...)
    int dr = 1;   // 特殊方格行坐标
    int dc = 1;   // 特殊方格列坐标

    // 初始化棋盘为 0
    for(int i=0; i<size; i++)
        for(int j=0; j<size; j++)
            board[i][j] = 0;

    chessBoard(0, 0, dr, dc, size);

    // 打印结果
    cout << "棋盘覆盖结果 (0 表示特殊方格):" << endl;
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            cout << setw(3) << board[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}

```

- 复杂度： $O(n^2)$ / $O(4^k)$

（重要）合并排序和快速排序

合并排序

- 基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。
- 实现：

```

int a[MAX_SIZE] = {};
int b[MAX_SIZE] = {};
void MergeSort(int a[], int left, int right) {
    if(left < right) {
        int middle = (left + right) / 2;
        // 分别排序

```

```

        MergeSort(a, left, middle);
        MergeSort(a, middle + 1, right);
        // 合并到b
        merge(a, b, left, middle, right);
        // 复制回a
        copy(a, b, left, right);
    }
}

void merge(int a[], int b[], int left, int middle, int right) {
    int i = left, j = middle + 1, k = left;

    while( i <= middle && j <= right) {
        if(a[i] <= a[j]) b[k++] = a[i++];
        else b[k++] = a[j++];
    }

    if(i > middle) {
        for(int rest = j; rest <= right; rest++) b[k++] = a[rest];
    }
    else {
        for(int rest = i; rest <= middle; rest++) b[k++] = a[rest];
    }
}

```

- 复杂度：
 - 时间： $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \cdot \log n)$
 - 左：规模变成一半，分成两部分
 - 右：合并结果只需要遍历一次即可
 - 最坏/平均都是该结果
 - 空间： $O(n)$

快速排序

- 基本思想：先根据一定规则进行分解，然后直接合并排好序的子结构，不需要额外的合并计算
- 步骤：
 1. 分解：根据基准元素，将数组划分成3段，分别是基准前的元素、基准元素、基准后的元素，要求前半部分都小于基准元素，后半部分都大于基准元素；
 2. 递归求解：对前后两部分子数组进行快速排序；
 3. 合并：由于前面设定的原则，不需要做多余操作
- 实现：

```

void QuickSort(int a[], int p, int right) {
    if(p < right) {
        int q = Partition(a, p, right);
        QuickSort(a, p, q - 1);
    }
}

```

```

        QuickSort(a, q + 1, right);
    }
}

int Partition(int a[], int p, int right) {
    int i = p, j = right + 1;
    int flag = a[p];

    while(1) {
        while(a[++i] < flag && i < right); // 找到左半部分大于flag的值
        while(a[--j] > flag); // 找到右半部分小于flag的值
        if(i >= j) break; // 若错过则符合要求，结束循环
        Swap(a, i, j); // 交换两个不符合要求的元素
    }

    a[p] = a[j];
    a[j] = flag;
    return j;
}

```

- 时间复杂度：
 - 最坏情况，有序数组一次只能分出一个元素： $T(n) = T(n - 1) + O(n) \Rightarrow O(n^2)$ （等差数列求和）
 - 最好情况，每次刚好分出一半： $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \cdot \log n)$
 - 平均情况：总体上只要划分不是极端不平衡，递归树的深度依然维持在 $O(\log n)$ 级别，即时间复杂度 $O(n \cdot \log n)$
- 改进：随机选择策略，即随机选择 `flag`，从而可以期望划分是较为对称的。

（重要）线性时间选择

- 问题描述：给定线性序集中 n 个元素和一个整数 k ， $1 \leq k \leq n$ ，要求找出这 n 个元素中第 k 小的元素。
- 基础：
 - 解决思路：类似快速排序，先进行随机划分，然后计算左半部分分区的长度 `left_len`，如果小于 k ，则目标存在于 `flag` 和右半分区中，接下来就转换成在剩下的部分中查找第 $n - k$ 小的元素，反之（大于等于 k ）则继续在左半分区中查找第 k 小的元素。
 - 实现：

```

int RandomizedSelect(int a[], int p, int r, int k) {
    if(p == r) return a[p];
    int i = RandomizedPartition(a, p, r);

    int left_len = i - p + 1;
    if(k <= left_len) return RandomizedSelect(a, p, i, k);
    else return RandomizedSelect(a, i + 1, r, k - left_len);
}

```

```

int Partition(int a[], int p, int r) {
    int rand_index = rand() % (r - p + 1) + p;
    Swap(a[p], a[rand_index]);

    int x = a[p];
    int i = p - 1;
    int j = r + 1;

    while (true) {
        // j 向左找小于 x 的
        do {
            j--;
        } while (a[j] > x);

        // i 向右找大于 x 的
        do {
            i++;
        } while (a[i] < x);

        if (i < j) {
            Swap(a[i], a[j]);
        }
        else {
            return j; // 分区结束，返回分界点
        }
    }
}

```

- 时间复杂度：
 - 最坏情况（每次去掉一个元素）： $O(n^2)$
 - 也可以可以在 $O(n)$ 平均时间内找出个输入元素中的第k小元素（每次大概对半分）
- 改进：
 - 解决思路：如果能在线性时间内找到一个优秀的划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 $\frac{1}{5}$ 倍（0到1），那么就可以在最坏情况下用 $O(n)$ 完成选择任务。
 - 解释：最坏情况下，若每次都可以缩短到至少 $\frac{1}{5}$ 倍，则计算时间满足 $T(n) \leq T(n \times \frac{1}{5}) + O(n)$ ，由于 $\frac{1}{5}$ 范围是从0到1，因此可得 $T(n) = O(n)$
 - 方法步骤：
 1. 将n个元素划分为 $\lceil n/5 \rceil$ 个组，每组5个元素（最后一组可能少于5个）；
 2. 对这些组分别排序（数量很小，因此看作常数复杂度），找到每组的中位数；
 3. 递归调用选择算法，找到这 $\lceil n/5 \rceil$ 个中位数的中位数（偶数则选择中位数中较大的），记为 x ，即为选定的划分基准；
 4. 根据 x 进行划分，进行后续操作。
 - 解释：
 - 问：为什么正确？

- 答：因为 x 为中位数的中位数，因此至少有一半的组 ($n/10$) 的中位数小于 x ，而同理每个组中都有2个元素小于当组的中位数，因此整体来看有 $3 * (n/10)$ 个元素小于 x （那些组的中位数也小于 x ），同理有至少 $3 * (n/10)$ 个元素大于 x ，这意味最坏情况也是 3:7 划分，实现了最坏情况也是 $O(n)$ 复杂度。
- 实现：

```
template <class Type>
Type Select(Type a[], int p, int r, int k) {
    // 步骤 1: 检查问题规模
    // 对应PDF: if (r-p < 75)
    // 当元素个数少于75时，直接排序返回结果，保证时间常数
    if (r - p < 75) {
        Sort(a, p, r);
        return a[p + k - 1];
    }

    // 步骤 2: 分组并收集每组的中位数
    // 对应PDF: 将n个输入元素划分为 n/5 个组，每组5个元素
    // 循环次数: (r - p - 4) / 5 即为组数 - 1
    for (int i = 0; i <= (r - p - 4) / 5; i++) {
        // 当前组的边界
        int left = p + 5 * i;
        int right = left + 4;

        // 对每组5个元素进行排序
        Sort(a, left, right);

        // 将每组的中位数（第3个元素）交换到数组前端 a[p+i] 的位置
        // 对应PDF图示: 将中位数提取出来
        Swap(a[p + i], a[left + 2]);
    }

    // 步骤 3: 递归寻找“中位数的中位数”
    // 对应PDF: 找出这 n/5 个中位数的中位数，作为划分基准 x
    // 中位数所在的范围是 a[p] 到 a[p + (r-p-4)/5]
    // 我们要找的是第 ((n/5)/2) 左右的元素
    Type x = Select(a, p, p + (r - p - 4) / 5, ((r - p - 4) / 5 + 1) / 2);

    // 步骤 4: 划分
    // 对应PDF: int i = Partition(a, p, r, x)
    // 使用 x 作为基准对数组进行划分
    int i = Partition(a, p, r, x);

    // 步骤 5: 尾递归求解
    // j 表示基准 x 在当前数组片段 a[p:r] 中的相对排名
    int j = i - p + 1;
```

```
// 对应PDF: if (k <= j) ... else ...
if (k <= j) {
    // 目标在左半段（包含基准）
    return Select(a, p, i, k);
} else {
    // 目标在右半段，寻找第 k-j 小的元素
    return Select(a, i + 1, r, k - j);
}
}
```

- 复杂度（ n 较大时；较小直接排序找，常数复杂度）： $T(n) \leq T(\frac{n}{5}) + T(\frac{3n}{4}) + Cn$
 - $T(\frac{n}{5})$: $\lceil n/5 \rceil$ 个组的中位数中找中位数用的也是 `Select`，规模为五分之一；
 - $T(\frac{3n}{4})$: 划分后剩余的规模大小，此处为宽松处理，正常可以是 $\frac{7n}{10}$ ；
 - 上面二者相加结果为 $\frac{1n}{20}$ ，保证了最终复杂度为 $O(n)$ 。

最接近点对问题

- 问题描述：给定平面 n 个点的集合 S ，找其中的一对点，使得在 n 个点组成的所有点对中，该点对之间的距离最小。
- 退化：一维情况，此时情况退化为 x 轴上的 n 个实数 $x_1, \dots, x_n$ ，找这 n 个实数中相差最小的两个实数。
 - 解决思路：
 - 主要：使用某个点 m 将 S 划分为 S_1 、 S_2 ，使用平衡子问题思想用 S 中各个点坐标的中位数来作分割点。递归找出最接近点 $\{p_1, q_1\}$ 和 $\{p_2, q_2\}$ ，并设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$ ，同时还有一个 $\{p_n, q_1\}$ ，最终结果为三者更小的一个。
 - 线性时间内找到 $\{p_n, q_1\}$ ：只有它们距离小于 d 的情况才需要，所以两个点到 m 的距离必须都不超过 d ，那么只需要在 $(m - d, m]$ 和 $(m, m + d]$ 两个区间各找到唯一点即可（因为左右区间单独的最小距离都是 d ，所有每个左开右闭的区间内至多有1个点）
 - 实现：

```
double CPair1(vector<double>& S) {
    int n = S.size();

    // 1. Base Case: 规模很小时，直接暴力求解
    // 为了安全性，n<2时返回无穷大
    if (n < 2) return DBL_MAX;
    if (n < 4) {
        double min_d = DBL_MAX;
        // 简单的两两比较  $O(n^2)$ ，但在 $n<4$ 时是常数时间  $O(1)$ 
        for (int i = 0; i < n; ++i) {
            for (int j = i + 1; j < n; ++j) {
                min_d = min(min_d, distance(S[i], S[j]));
            }
        }
        return min_d;
    }
}
```

```

}

// 2. Divide: 划分
// 对应课件: m = S中各点坐标的中位数; 构造S1, S2...
// 使用 nth_element 在 O(n) 时间内找到中位数并进行划分
// 这保证了 S1 和 S2 的大小都约为 n/2, 满足递归方程
auto mid_iterator = S.begin() + n / 2;
std::nth_element(S.begin(), mid_iterator, S.end());

// 构造 S1 (左半部分) 和 S2 (右半部分)
// S1 包含 [begin, mid)
// S2 包含 [mid, end)
vector<double> S1(S.begin(), mid_iterator);
vector<double> S2(mid_iterator, S.end());

// 3. Conquer: 递归求解
// 对应课件: d1 = cpair1(S1); d2 = cpair1(S2);
double d1 = CPair1(S1);
double d2 = CPair1(S2);

// 取左右两边最小距离的较小值
double d = min(d1, d2);

// 4. Merge: 合并求解跨越分割点的对
// 对应课件: p = max(S1); q = min(S2); d = min(d1, d2, q-p);

// 在 S1 中找最大值 (最靠近分割线左侧的点)
double p = *max_element(S1.begin(), S1.end());

// 在 S2 中找最小值 (最靠近分割线右侧的点)
double q = *min_element(S2.begin(), S2.end());

// 计算跨界距离 q - p, 并更新最终结果
// 注意: 因为nth_element保证了右边的元素 >= 左边的元素, 所以 q >= p
return min(d, q - p);
}

```

- 复杂度: $T(n) = 2T(n/2) + O(1)$, 但是还要先排序, 所以最终复杂度为排序复杂度 $O(n \cdot \log n)$ 。
- 二维解决思路:
 - 主要: 选取一垂直线 $l: x = m$ 作为分割直线, 其中 m 为 S 中各点坐标的中位数, 由此 $S \rightarrow S_1 \quad S_2$, 递归的在两个分区中找到最小的距离 d_i 并设 $d = \min\{d_1, d_2\}$, 然后同样只需要找到分布于临界边两侧的最小距离点对 $\{p, q\}$, 然后比较找最小即可。
 - 线性时间找 $\{p, q\}$: 对于 S_1 中任意点 p , 若有 $distance(p, q) < d$, 则这个点 q 一定落在一个大小为 $d \times 2d$ 的矩形 R 中 (p 的纵坐标 $+d/-d$, 横向为 $m + d$), 同时由鸽巢原理可以证明矩形中最多只有6个 S 中的点 (反证法: 平均分6块, 每块对角线为 $\frac{25}{36}d$ 小于 d , 不符合), 因此在合并步骤中最多只需要检查 $6 \times n/2 = 3n$ 个候选者。接下

来确定检查哪些点，需要将所有点投射到垂直线 l 上（只看 y 坐标），找到所有和 p 投影的距离少于 d 的投影的点，即将所有 S 中的点按照 y 坐标排序，然后对 P_1 中所有点对排好序的序列做一次扫描，就可以找到所有候选者，此时只需要检查它后面的6个点就可以覆盖所有可能的最接近点对（前面6个点中，当前的点属于被选择的，已经被检查过）。

- 实现：

```
// 定义点结构，增加 type 字段严格对应 P1 和 P2 的区分
struct Point {
    double x, y;
    int type; // 0 表示来自 P1（左侧），1 表示来自 P2（右侧）
};

double ScanAndCheck6Points(vector<Point>& P1, vector<Point>& P2, double
dm) {
    // Step 1: 将 P1 和 P2 中所有点按其 y 坐标排好序"
    vector<Point> strip;
    strip.reserve(P1.size() + P2.size());

    // 将 P1 的点加入，标记为 0
    for (auto& p : P1) {
        p.type = 0;
        strip.push_back(p);
    }
    // 将 P2 的点加入，标记为 1
    for (auto& p : P2) {
        p.type = 1;
        strip.push_back(p);
    }

    // Step 2: 排序
    // 对 strip 进行 Y 轴排序
    sort(strip.begin(), strip.end(), compareY);

    // 当前找到的跨区域最小距离，初始化为 dm
    double min_dist = dm;

    // Step 3: 线性扫描
    for (int i = 0; i < strip.size(); ++i) {
        // 向后检查紧随的邻居点
        for (int j = i + 1; j < i + 7; ++j) {

            // 【几何剪枝 - 核心】
            // 如果 Y 轴距离差已经 >= min_dist，后面的点都不用看了
            // 根据 P62 的鸽舍原理，这个 break 保证了内层循环最多执行常数次（约6
            // 次）

            if (strip[j].y - strip[i].y >= min_dist) {
                break;
            }
        }
    }
}
```

```

        // 【逻辑剪枝 - 对应 P63 文本】
        // 课件提到 "P1 中每一点检查 P2 中..."
        // 这意味着同属 P1 或同属 P2 的点对不需要计算（因为递归时已经算过了，
        且肯定 >= dm）
        // 只有当两个点 type 不同时（一个左一个右），才计算距离
        if (strip[i].type != strip[j].type) {
            double d = dist(strip[i], strip[j]);
            if (d < min_dist) {
                min_dist = d;
            }
        }
    }
}

return min_dist;
}

double cpair2(S) {
    n = |S|;
    if (n < 4) return BruteForce(S); // 1. Base Case

    // 2. Divide
    m = S.median_X;
    S1 = {p in S | p.x <= m};
    S2 = {p in S | p.x > m};

    // 3. Conquer
    d1 = cpair2(S1);
    d2 = cpair2(S2);
    dm = min(d1, d2);

    // 4. Merge
    // 构造带状区域点集 P1, P2（距分割线距离 < dm）
    // 按 Y 坐标排序这些点
    // 扫描：对每个点，检查后续 6 个点的距离
    dl = ScanAndCheck6Points(P1, P2, dm);

    d = min(dm, dl);
    return d;
}

```

- 复杂度： $T(n) = 2T(n/2) + O(n)$, $n \geq 4$, 即 $T(n) = O(n \cdot \log n)$ 。

循环赛日程表

- 问题描述：设计满足下列要求的比赛日程表：(1) 每个选手必须与其他 $n-1$ 个选手各赛一次；(2) 每个选手一天只能赛一次；(3) 循环赛一共进行 $n-1$ 天。输入为 k ，表示有 2^k 个

选手，输出为一个 $n \times n$ 的二维数组，第*i*行为选手*i*，第*j*列为第*j*-1天的对手（*j*从1开始，第一列填自己）

- 解决思想：将所有选手分为两半，*n*个选手的日程表可以通过为两个*n*/2个选手的组设计好的日程表决定，分割直到只剩2个选手时只需要让这两个选手比赛即可。
- 实现：

```
void Table(int k, int** a)
{
    // 1. 计算总人数  $n = 2^k$ 
    int n = 1;
    for (int i = 1; i <= k; i++) n *= 2;

    // 保存原始的 N 用于后续逻辑核对
    int total_n = n;

    // 2. 初始化第一行
    for (int i = 1; i <= n; i++) a[1][i] = i;

    // m: 当前已经构造好的块的大小，可以被复制的
    int m = 1;

    // 3. 构造过程
    for (int s = 1; s <= k; s++) {
        n /= 2; // 确定有几个大块，也就是说  $n / m$ ，确定需要完成多少次复制操作
        for (int t = 1; t <= n; t++) { // 开始复制操作，决定次数
            // 每一个大块需要复制  $m*m$  个元素，[1][1]到[m][m]是正确的内容
            for (int i = m + 1; i <= 2 * m; i++) {
                for (int j = m + 1; j <= 2 * m; j++) {
                    // 核心赋值逻辑
                    // (t-1)*m*2 确定当前处理的是第几个大块
                    // 右下 = 左上
                    a[i][j + (t - 1) * m * 2] = a[i - m][j + (t - 1) * m
* 2 - m];

                    // 左下 = 右上
                    a[i][j + (t - 1) * m * 2 - m] = a[i - m][j + (t - 1)
* m * 2];

                    // 上面的区别就在于 -m 的位置
                }
            }
        }
        m *= 2;
    }

    // 4. 打印结果
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= m; j++) {
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
}
```

```
}  
}
```

- 复杂度： $O(n^2)$ ，本质上是进行了一次有规则的、无重复的二维数组赋值，因此就是一次遍历 $n \times n$ 数组的过程。

动态规划

基础

概念

- 总体思想：分治法，将待求解问题分解成若干的子问题
- 特色：分解得到的子问题并不是相互独立的，而是相互依赖的。
- 解决问题：分治方法得到的不同子问题数目通常只有多项式量级，但是有些子问题会被重复计算多次，浪费资源。
- 解决方法：空间换时间。即保存已解决的子问题的答案，在需要的时候找出记录的答案，可以避免大量重复计算，从而得到多项式时间的算法。

基本要素

- **最优子结构性质**：整体最优解包含着其子问题的最优解
 - 分析：反证法，即假设原问题最优解导出的子问题最优解不是最优的，然后说明新的子问题最优解可以构造出原问题的更优解，从而证伪
- **重叠子问题性质**：递归算法求解时产生的子问题并不总是新问题，导致有的子问题重复计算多次
- **备忘录方法**：控制结构与直接递归的控制结构相同，但是为每个计算过的子问题建立备忘录以备需要时查看，避免相同子问题的重复求解

步骤

1. 找出最优解的性质，并刻画其结构特征
2. 递归地定义最优值
3. 以**自底向上**的方式计算出最优值
4. 根据计算最优值时得到的信息，构造最优解

实例

矩阵连乘问题

- 基础知识：
 - 完全加括号的矩阵连乘积递归的定义：
 1. 单个矩阵是完全加括号的、
 2. 矩阵连乘积 A 是完全加括号的，则 A 可以表示为两个完全加括号的矩阵连乘积 B 和 C 的乘积并加括号，即 $A = (BC)$

- 问题描述：给定n个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i, A_{i+1} 是可乘的，考察这n个矩阵的连乘积 $A_1 A_2 \dots A_n$ ，如何确定计算矩阵连乘积的计算次序使得依次序计算矩阵连乘积的数乘次数最少。
- 原方法：穷举法

算法复杂度分析：

对于n个矩阵的连乘积，设其不同的计算次序为 $P(n)$ 。
由于每种加括号方式都可以分解为两个子矩阵的加括号问题：
 $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ 可以得到关于 $P(n)$ 的递推式如下：

$$P(n) = \begin{cases} 1 & n=1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n>1 \end{cases} \Rightarrow P(n) = \Omega(4^n / n^{3/2})$$

- DP解决思路：
 - 将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 记为 $A[i:j]$ ，若从k断开连乘，则相应的计算量变化为： $A[i:j] \rightarrow A[i:k] + A[i+1:j] + (A[i:k] \text{ 和 } A[i+1:j] \text{ 相乘的计算量})$
 - 特征：计算 $A[i:j]$ 最优次序包含的计算子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的
 - 建立递归关系：
 - 记 $A[i:j]$ 的最少的乘的次数为 $m[i, j]$ ，则原问题最优解为 $m[1, n]$
 - 当 $i = j$ ， $A[i:j] = A_i$ ，即 $m[i, i] = 0$ （最小子问题）
 - 当 $i < j$ ，设 A_i 的列数为 p_i ，则 $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$
- 实现：

```
/*
p: 列数数组
n: 矩阵数
m: 最小数乘数量数组
s: 分割点记录
*/
void MatrixChain(int *p, int n, int **m, int **s) {
    // 一个矩阵最佳结果
    for (int i = 1; i <= n; i++) m[i][i] = 0;

    // 自底向上开始计算，r 表示 r 个矩阵相乘的最少数乘次数
    for (int r = 2; r <= n; r++) {
        // i 代表起始位置，即左界
        for (int i = 1; i < n - r + 1; i++) {
            int j = i + r - 1; // 确定右界
            m[i][j] = m[i + 1][j] + p[i - 1] * p[i] * p[j]; // 设分为
            // [i:i] 和 [i+1:j]，省略0
            s[i][j] = i;

            // 遍历找分割点
            for (int k = i + 1; k < j; k++) {
                int tmp = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] *
```

```

p[j];

        if (tmp < m[i][j]) {
            // 更新
            m[i][j] = tmp;
            s[i][j] = k;
        }
    }
}
}
}
}

```

- 复杂度：
 - 时间：主要计算量为对分组长度 r 、起始位置 i 、分割位置 k 的三重循环，因此计算时间上界为 $O(n^3)$
 - 空间： $O(n^2)$

（重要）最长公共子序列

- 基础定义：
 - 子序列：给定序列 $X = \{x_1, x_2, \dots, x_m\}$ ，另一序列 $Z = \{z_1, z_2, \dots, z_k\}$ 。若 Z 是 X 的子序列，是指存在一个严格递增下标序列 $\{i_1, i_2, \dots, i_k\}$ ，使得对于所有 $j = 1, 2, \dots, k$ 有： $z_j = x_{i_j}$ 。
 - 公共子序列：给定 2 个序列 X 和 Y ，当另一序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的公共子序列。
- 问题描述：给定 2 个序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ ，找出 X 和 Y 的最长公共子序列。
- 解决思路（从后往前看）：
 - 结构：设序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z = \{z_1, z_2, \dots, z_k\}$ ，则
 1. 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的最长公共子序列。
 2. 若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，则 Z 是 X_{m-1} 和 Y 的最长公共子序列。
 3. 若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，则 Z 是 X 和 Y_{n-1} 的最长公共子序列。
 - 由此可见，两个序列的最长公共子序列包含这两个序列的前缀的最长公共子序列，即该问题具有最优子结构性质。
 - 递归结构：用 $c[i][j]$ 记录序列 X_i 和 Y_j 的最长公共子序列的长度。总结递归结构如下：

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

- 实现：

```

/*
m, n: 原序列长度
x, y: 原序列

```

c: 记录x前i个和y前j个的最长子序列长度

b: 存储操作类型, 方便后续LCS构造

*/

```
void LCSLength(int m, int n, char *x, char *y, int **c, int **b) {
    int i, j;

    // 初始化
    for (i = 0; i <= m; i++) c[i][0] = 0;
    for (j = 0; j <= n; j++) c[0][j] = 0;

    // 遍历 c 填充结果
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) {
            if (x[i] == y[j]) {
                c[i][j] = c[i - 1][j - 1] + 1;
                b[i][j] = 1;
            }
            else if (c[i - 1][j] >= c[i][j - 1]) {
                c[i][j] = c[i - 1][j];
                b[i][j] = 2;
            }
            else {
                c[i][j] = c[i][j - 1];
                b[i][j] = 3;
            }
        }
    }
}
```

// 构造最长公共子序列

```
void LCS(int i, int j, char *x, int **b) {
    if (i == 0 || j == 0) return;

    if (b[i][j] == 1) {
        LCS(i - 1, j - 1, x, b);
        cout << x[i];
    }
    else if (b[i][j] == 2) {
        LCS(i - 1, j, x, b);
    }
    else if (b[i][j] == 3) {
        LCS(i, j - 1, x, b);
    }
}
```

- 改进:

- 省略数组b: 直接通过判断 `x[i] ? x[j]` 和 `c[i-1][j] ? c[i][j-1]` 的结果来决定下一步, 可以省掉 $O(m*n)$ 的空间。
- 压缩数组c (只计算最大子序列长度):

- 只开两行 n 列，一行用来记录当前的 i ，另一行记录 $i-1$ ， $O(2 \times n)$ ；
- 只开一个一维数组，其中 $dp[j]$ 代表 $c[i-1][j]$ ， $dp[j-1]$ 代表 $c[i][j-1]$ ，额外维护一个变量 $prev = c[i-1][j-1]$ ，遍历时更新 $dp[j]$ ， $O(n)$ 。

```
int LCS_RollingArray(string text1, string text2) {
    int m = text1.length();
    int n = text2.length();

    // 【优化技巧】
    // 确保 text2 是较短的那个字符串，这样开辟的数组 dp 空间更小
    // 空间复杂度降为  $O(\min(m, n))$ 
    if (m < n) {
        swap(text1, text2);
        swap(m, n);
    }

    // 只开辟一维数组，初始化为 0
    vector<int> dp(n + 1, 0);

    // 外层循环遍历 text1（相当于行）
    for (int i = 1; i <= m; i++) {
        int prev = 0; // prev 记录的是“左上角”的值 ( $c[i-1][j-1]$ )，
        // 对于每行第一个元素，左上角总是 0
        // 内层循环遍历 text2（相当于列）
        for (int j = 1; j <= n; j++) {
            // temp 暂存更新前的 dp[j]，因为更新后的 dp[j] 会变成新一
            // 行的值，但下一次循环(j+1)需要用到现在的 dp[j] 作为它的  $c[i-1][j-1]$ 
            int temp = dp[j];
            if (text1[i - 1] == text2[j - 1]) {
                dp[j] = prev + 1;
            }
            else {
                dp[j] = max(dp[j], dp[j - 1]);
            }

            // 更新 prev
            prev = temp;
        }
    }
    return dp[n];
}
```

- 复杂度：
 - 时间：两层循环 $O(m \times n)$
 - 空间： $O(m \times n) \rightarrow O(2 \times n) \rightarrow O(n)$

（重要）最大子段和

- 基础定义/问题描述：

- 给定由n个整数（可能为负整数）组成的序列 $a_1, a_2, \dots, a_n$ ，求该序列形如 $\sum_{k=i}^j a_k$ 的子段和的最大值。
- 当所有整数均为负整数时，定义其最大子段和为0。
- 所求的最优值为 $\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$
- 简单算法：遍历 i from 1 to n-1 & j from i+1 to n，找最大值，时间复杂度 $O(n^2)$ 。
- 分治算法：将数组一分为2，结果有三种情况：一是等于左半部分；二是等于右半部分；三是在中间，中间的情况只需要算出左半部分以末尾元素结尾的最大字段和和右半部分开头元素为开头的最大字段和（以固定元素为起始遍历一次子数组找最大值，所以额外开销为 $O(n)$ ），然后相加就是第三种情况的最大值，最后取三种情况中的最大值即可。复杂度为 $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \cdot \log n)$
- DP解决思路：选定最大子段和的最后一个元素 $a[j]$ ，要求必须包含该元素，让该条件下的最大子段和为 $b[j]$ ，那么很显然我们要找的最大子段和就是数组 b 中最大的一个。同时，计算过程中显然可以通过 $b[j-1]$ 来计算 $b[j]$ ，若 $b[j-1] > 0$ ，那么由于必须包含 $a[j]$ ，直接相加就是 $b[j]$ ，反之，前面为负数被定义强制为0，那么 $b[j]$ 就等于 $0 + a[j] = a[j]$ 。总结后严格定义如下：
 - 若 $b[j] = \max_{1 \leq i \leq j} \{ \sum_{k=i}^j a[k] \} (1 \leq j \leq n)$ ，则所求的最大子段和为

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} b[j]$$

- 由 $b[j]$ 的定义易知，当 $b[j-1] > 0$ 时， $b[j] = b[j-1] + a[j]$ ，否则 $b[j] = a[j]$ ，由此可得，计算 $b[j]$ 的动态规划递归式

$$b[j] = \max\{b[j-1] + a[j], a[j]\} \quad 1 \leq j \leq n$$

- 实现：

```
int MaxSum(int n, int *a) {
    // 最小值都为0
    int sum = 0, b = 0;

    // 遍历所有结尾元素
    for (int j = 1; j <= n; j++) {
        // 递归式子，将数组归为元素，因为递归式中只需要 b[j-1]，没必要浪费空间
        if (b > 0) b += a[j];
        else b = a[j];

        // 更新最终结果
        if (b > sum) sum = b;
    }
    return sum;
}
```

- 复杂度：
 - 时间： $O(n)$

- 空间： $O(1)$
- 推广：
 - 最大子矩阵和问题
 - 问题描述： $m \times n$ 矩阵中，找一个子矩阵，要求元素和最大。
 - 解决思路：基本思路就是通过确定 i_1, i_2 确定行的范围，将二维转成一维的最大子段和问题，具体内容为定义最大子矩阵和问题的最优值为：

$$\max_{\substack{1 \leq i_1 \leq i_2 \leq m \\ 1 \leq j_1 \leq j_2 \leq n}} s(i_1, i_2, j_1, j_2) = \max_{\substack{1 \leq i_1 \leq i_2 \leq m \\ 1 \leq j_1 \leq j_2 \leq n}} \sum_{i=i_1}^{i_2} \sum_{j=j_1}^{j_2} a[i][j] = \max_{1 \leq i_1 \leq i_2 \leq m} t(i_1, i_2)$$

式中，

$$t(i_1, i_2) = \max_{1 \leq j_1 \leq j_2 \leq n} \sum_{i=i_1}^{i_2} \sum_{j=j_1}^{j_2} a[i][j]$$

设 $b[j] = \sum_{i=i_1}^{i_2} a[i][j]$ ，则

$$t(i_1, i_2) = \max_{1 \leq j_1 \leq j_2 \leq n} \sum_{j=j_1}^{j_2} b[j]$$

- 实现：

```
int MaxSum2(int m, int n, int **a) {
    int sum = 0;
    // 用于存储转化后的一维数组
    int *b = new int[n + 1];

    // 遍历边界，复杂度  $O(m^2)$ 
    for (int i1 = 1; i1 <= m; i1++) {
        // 先置空
        for (int j = 1; j <= n; j++) b[j] = 0;
        for (int i2 = i1; i2 <= m; i2++) {
            // 压缩过程
            for (int j = 1; j <= n; j++) b[j] += a[i2][j];
            // 求最大子段和
            int max = MaxSum(n, b);
            // 更新结果
            if (max > sum) sum = max;
        }
    }
}
```

- 时间复杂度：外层遍历范围用了 $O(m^2)$ ，每次敲定范围后求最大子段和用 $O(n)$ ，所以时间复杂度为 $O(m^2n)$ 。
- 最大m字段和问题：
 - 问题描述：找m个不相交子段，使得m个子段总和最大。
 - 解决思路：

- 设 $b(i, j)$ 表示数组的前 j 项中 i 个字段和的最大值，且第 i 个子段必须包含 $a[j]$ ，显然最优值为 $\max_{m \leq j \leq n} b(m, j)$
- 递归关系式：
 - 递归： $b(i, j) = \max\{b(i, j-1) + a[j], \max_{i-1 \leq t < j} b(i-1, t) + a[j]\}$
 - 情况1：直接把 $a[j]$ 接到上一个结果的第 i 个子段后面
 - 情况2：在前面的情况中选出 $i-1$ 个子段，然后 $a[j]$ 单独成为一个子段
 - 初始： $b(0, j) = 0, b(i, 0) = 0$

- 实现

```
int MaxSumM(int m, int n, int *a) {
    // 异常情况处理：如果数组长度小于段数，或段数小于1，返回0
    if (n < m || m < 1) return 0;

    // 动态申请二维数组 b[m+1][n+1]
    // b[i][j] 表示：在前 j 个数中取 i 个子段，且第 i 个子段以 a[j] 结尾的最大和
    int **b = new int *[m + 1];
    for (int i = 0; i <= m; i++) {
        b[i] = new int[n + 1];
        b[i][0] = 0; // 边界初始化
    }

    // 核心循环：i 表示当前正在算第几段
    for (int i = 1; i <= m; i++) {
        // j 表示当前元素的位置
        // 优化边界：j 至少是 i（每个段至少1个元素），
        // j 最多是 n - (m - i)（要给后面剩下的 m-i 个段预留至少 m-i 个位置）
        for (int j = i; j <= n - m + i; j++) {

            if (j > i) {
                // 情况 1：将 a[j] 接在当前第 i 段的末尾
                b[i][j] = b[i][j - 1] + a[j];

                // 情况 2：a[j] 作为一个新段的开始（第 i 段只有 a[j]）

                // 需要遍历上一段 (i-1) 可能的结束位置 k，找一个最大的
                for (int t = i - 1; t < j; t++) {
                    if (b[i][j] < b[i - 1][t] + a[j]) {
                        b[i][j] = b[i - 1][t] + a[j];
                    }
                }
            } else {
                // 边界情况：当 j == i 时，每个元素必须各成一段
                b[i][j] = b[i - 1][j - 1] + a[j];
            }
        }
    }
}
```

```

    }
}

// 寻找结果：最后一段（m段）可以结束在 m 到 n 的任意位置，取最大值
int sum = 0;
for (int j = m; j <= n; j++) {
    if (sum < b[m][j]) {
        sum = b[m][j];
    }
}

return sum;
}

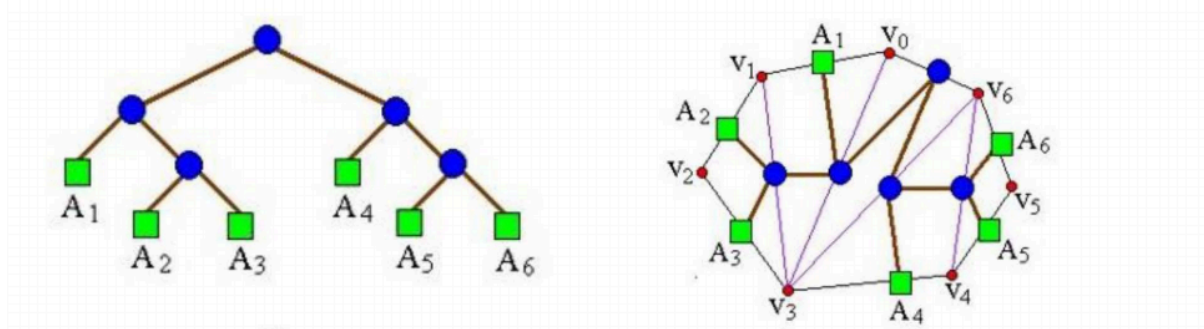
```

- 复杂度
 - 时间： $O(mn^2)$
 - 空间： $O(mn)$

凸多边形最优三角剖分

- 基本概念：
 - 多边形定点的逆时针序列表表示多边形，即 $P = \{v_0, v_1, \dots, v_{n-1}\}$ 。
 - 多边形的三角剖分：将多边形分割成互不相交的三角形的弦的集合 T 。
- 问题描述：给定凸多边形 P ，以及定义在由多边形的边和弦组成的三角形上的权函数 w ，要求确定凸多边形的三角剖分，使得该三角剖分中各个三角形上权之和为最小。
- 解决思路：转换成矩阵连乘问题：

• 矩阵连乘积中的每个矩阵 A_i 对应于凸 $(n+1)$ 边形中的一条边 $v_{i-1}v_i$ 。三角剖分中的一条弦 $v_i v_j$ ， $i < j$ ，对应于矩阵连乘积 $A[i+1:j]$ 。



- 最优子结构性质：若凸 $(n+1)$ 边形 $P = \{v_0, v_1, \dots, v_n\}$ 的最优三角剖分 T 包含三角形 $v_0 v_k v_n$ ， $1 \leq k \leq n-1$ ，则 T 的权为 3 个部分权的和：三角形 $v_0 v_k v_n$ 的权，子多边形 $\{v_0, v_1, \dots, v_k\}$ 和 $\{v_k, v_{k+1}, \dots, v_n\}$ 的权之和。可以断言，由 T 所确定的这 2 个子多边形的三角剖分也是最优的。因为若有 $\{v_0, v_1, \dots, v_k\}$ 或 $\{v_k, v_{k+1}, \dots, v_n\}$ 的更小权的三角剖分将导致 T 不是最优三角剖分的矛盾。
- 递归结构：类似矩阵连乘问题。

$$t[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{t[i][k] + t[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$$

- 实现：

```
void MinWeightTriangulation(int n, Type **t, int **s)
{
    // 初始化：长度为1的退化多边形（即一条边）权值为0
    for (int i = 1; i <= n; i++) t[i][i] = 0;

    // r 表示当前计算的子多边形的规模（涉及的顶点跨度）
    // r=2 代表计算两个相邻矩阵（或三角形的底边跨度）的情况
    for (int r = 2; r <= n; r++) /* 多边形节点个数为r+1 */
    {
        // i 是子多边形的起始位置
        for (int i = 1; i <= n - r + 1; i++) {
            // j 是子多边形的结束位置
            int j = i + r - 1;

            // --- 核心状态转移逻辑 ---

            // 1. 先假设分割点 k = i, 作为当前的初始最小值
            // t[i][i] 为 0, 所以省略不写, 公式原形为 t[i][i] + t[i+1][j] + w
            t[i][j] = t[i+1][j] + w(i-1, i, j);
            s[i][j] = i; // 记录分割点

            // 2. 遍历剩余的分割点 k (从 i+1 到 j-1)
            for (int k = i + 1; k < i + r - 1; k++) { /* j = i+r-1 */
                // 计算分割点为 k 时的总权值
                // u = 左半部分最优解 + 右半部分最优解 + 形成的三角形(i-1, k,
                // j)的权值

                int u = t[i][k] + t[k+1][j] + w(i-1, k, j);

                // 如果找到更小的权值, 更新最优解和分割点
                if (u < t[i][j]) {
                    t[i][j] = u;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

- 复杂度：同矩阵连乘

多边形游戏

- 问题描述：开始有一个由n个顶点构成的多边形。每个顶点被赋予一个整数值，每条边赋予一个运算符（+/*），所有边用整数从1到n编号。游戏第一步将一条边删除，随后n-1步按照以下方式操作：
 - 选择一条边E以及其两个顶点V1和V2；

- 用一个新的顶点取代边E以及由E连接的两个顶点V1和V2。将 $V1 \text{ op } V2$ 结果赋给新的顶点；
- 最后所有边被删除，要求最后一个点的值最大。
- 解决思路：
 - 在所给多边形中，从顶点 $i(1 \leq i \leq n)$ 开始，长度为 j (链中有 j 个顶点)的顺时针链 $p(i, j)$ 可表示为： $v[i], \text{op}[i+1], v[i+1], \text{op}[i+2], \dots, v[i+j-1]$ 。
 - 如果这条链的最后一次合并运算在 $\text{op}[i+s]$ 处发生($1 \leq s \leq j-1$)，则可在 $\text{op}[i+s]$ 处将链分割为2个子链 $p(i, s)$ 和 $p(i+s, j-s)$ 。
 - 设 $m1$ 是对子链 $p(i, s)$ 的任意一种合并方式得到的值，而 a 和 b 分别是在所有可能的合并中得到的最小值和最大值。 $m2$ 是 $p(i+s, j-s)$ 的任意一种合并方式得到的值，而 c 和 d 分别是在所有可能的合并中得到的最小值和最大值。依此定义有 $a \leq m1 \leq b$, $c \leq m2 \leq d$
 - 当 $\text{op}[i+s] = '+'$ 时，显然有 $a + c \leq m \leq b + d$
 - 当 $\text{op}[i+s] = '*'$ 时，有 $\min\{ac, ad, bc, bd\} \leq m \leq \max\{ac, ad, bc, bd\}$
 - 换句话说，主链的最大值和最小值可由子链的最大值和最小值得到。
 - 递归公式如下：

$$\min f(i, j, s) = \begin{cases} a + c & \text{op}[i+s] = '+' \\ \min\{ac, ad, bc, bd\} & \text{op}[i+s] = '*' \end{cases}$$

$$\max f(i, j, s) = \begin{cases} b + d & \text{op}[i+s] = '+' \\ \max\{ac, ad, bc, bd\} & \text{op}[i+s] = '*' \end{cases}$$

- 由于最优断开位置 s 有 $1 \leq s \leq j-1$ 的 $j-1$ 种情况，由此可设（最后一位维度为2，用于表示记录最小值和最大值）：

$$m[i, j, 0] = \min_{1 \leq s \leq j} \{ \min f(i, j, s) \} \quad 1 \leq i, j \leq n$$

$$m[i, j, 1] = \max_{1 \leq s \leq j} \{ \max f(i, j, s) \} \quad 1 \leq i, j \leq n$$

$$\text{初始边界值: } m[i, 1, 0] = v[i] \quad m[i, 1, 1] = v[i]$$

- 实现：

```
void MinMax(int n, int i, int s, int j, int &minf, int &maxf) {
    // 1. 确定右半部分的起始位置（处理环形结构）
    // i 是左半部分起点，s 是左半部分长度
    // (i + s - 1) % n + 1 是为了保持 1-based 索引（1到n）
    int r = (i + s - 1) % n + 1;

    // 2. 获取左右两部分已计算出的最大/最小值
    // 左部分：起点 i，长度 s
    int minL = m[i][s][0];
    int maxL = m[i][s][1];

    // 右部分：起点 r，长度 j-s
    int minR = m[r][j - s][0];
    int maxR = m[r][j - s][1];
```

```

// 3. 根据连接两部分的运算符进行计算
// 这里假设 op[r] 是连接左尾和右头的运算符
char operatorType = op[r];

if (operatorType == '+') {
    // 加法比较简单：最大+最大，最小+最小
    maxf = maxL + maxR;
    minf = minL + minR;
}
else if (operatorType == '*') {
    // 乘法比较复杂，因为负负得正
    // 需要计算四种组合
    int a = maxL * maxR;
    int b = maxL * minR;
    int c = minL * maxR;
    int d = minL * minR;

    // 从这四个值中找出最大和最小
    maxf = max(max(a, b), max(c, d));
    minf = min(min(a, b), min(c, d));
}
}

int PolyMax(int n) {
    int minf, maxf;
    m[i][1][0] = m[i][1][1] = v[i];
    for (int j = 2; j <= n; j++) {
        for (int i = 1; i <= n; i++) {
            for (int s = 1; s < j; s++) {
                MinMax(n, i, s, j, minf, maxf); // 计算当前分割下的潜在最小/
最大值

                if (m[i][j][0] > minf)
                    m[i][j][0] = minf;
                if (m[i][j][1] < maxf)
                    m[i][j][1] = maxf;
            }
        }
    }
    int temp = m[1][n][1];
    for (int i = 2; i <= n; i++) {
        if (temp < m[i][n][1])
            temp = m[i][n][1];
    }
    return temp;
}

```

- 复杂度：三套循环 $O(n^3)$ 。

图像压缩

- 问题描述：

- 图象的变位压缩存储格式将所给的像素点序列 $\{p_1, p_2, \dots, p_n\}$, $0 \leq p_i < 255$ 分割成 m 个连续段 $S_1, S_2, \dots, S_m$ 。第 i 个像素段 S_i 中 ($1 \leq i \leq m$)，有 $l[i]$ 个像素，且该段中每个像素都只用 $b[i]$ 位表示。设 $t[i] = \sum_{k=1}^{i-1} l[k]$ ，则第 i 个像素段 S_i 为 $S_i = \{p_{t[i]+1}, \dots, p_{t[i]+l[i]}\}$ 。设 $h_i = \lceil \log(\max_{t[i]+1 \leq k \leq t[i]+l[i]} p_k + 1) \rceil$ ，则 $h[i] \leq b_i \leq 8$ 。因此需要用 3 位表示 b_i 。如果限制 $1 \leq l_i \leq 255$ ，则需要用 8 位表示 l_i 。因此，第 i 个像素段所需的存储空间为 $l[i] \cdot b[i] + 11$ 位。按此格式存储像素序列 $\{p_1, p_2, \dots, p_n\}$ ，需要 $\sum_{i=1}^m (l_i \cdot b_i + 11)$ 位。
- 图象压缩问题要求确定像素序列 $\{p_1, p_2, \dots, p_n\}$ 的最优分段，使得依此分段所需的存储空间最少。每个分段的长度不超过 256 位。

- 解决方法：

- 设 $s[i]$ 为像素序列 $\{p_1, p_2, \dots, p_i\}$ 的最优分段所需的存储位数。则有 $s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * bmax(i-k+1, i)\} + 11$ ，其中 $bmax(i, j) = \lceil \log(\max_{i \leq k \leq j} \{p_k\} + 1) \rceil$ 。
- 遍历 k 为最后一段的长度，找到最后一段为多少使得当前子序列的所需位数最少
- 使用前面 $i-k$ 序列的最小位数加上当前分割方法需要的最后一段的位数 (k 个像素，每个像素最少需要 $bmax(i-k+1, i)$ 位)。
- 确认后，需要额外的 3 (找出的最后一段存储使用的位数) + 8 (找出的最后一段中像素个数) = 11 存储相关信息。

- 实现：

```
// p: 像素序列、s: DP数组、l: 以第i个像素结尾的那一段包含的像素个数、b: 存第i个像素的位数
void Compress(int n, int p[], int s[], int l[], int b[]) {
    int Lmax = 256, header = 11;
    s[0] = 0;
    for (int i = 1; i <= n; i++) {
        // 先默认最后一段的长度为1
        b[i] = length(p[i]);
        int bmax = b[i];
        s[i] = s[i-1] + bmax; // * 1;
        l[i] = 1;
        for (int k = 2; k <= i && k <= Lmax; k++) {
            if (bmax < b[i-k+1]) bmax = b[i-k+1];
            if (s[i] > s[i-k] + k * bmax) {
                s[i] = s[i-k] + k * bmax;
                l[i] = k;
            }
        }
        s[i] += header;
    }
}
```

```
}  
}
```

- 复杂度：正常分析为 $O(n \cdot Lmax)$ ，但是由于后者为常数，因此时间复杂度为 $O(n)$ 。

电路布线

- 问题描述：在一块电路板的上下两端分别有 n 个接线柱，要求用导线 $(i, \pi(i))$ 将上下端接线柱相连。其中 $\pi(i)$ 是 $\{1, 2, \dots, n\}$ 的一个排列。导线 $(i, \pi(i))$ 称为该电路板上的第 i 条连线。对任何 $1 \leq i \leq j \leq n$ ，第 i 条连线和第 j 条连线相交的充分且必要的条件是 $\pi(i) > \pi(j)$ 。要求确定将哪些连线安排在第一层上，使得该层上有尽可能多的连线。即，要求确定导线集 $Nets = \{(i, \pi(i)), 1 \leq i \leq n\}$ 的最大不相交子集。（最长上升子序列）

- 解决方法：

- $N(i, j) = \{t | (t, \pi(t)) \in Nets, t \leq i, \pi(t) \leq j\}$ ，分别是上下端的最大边界
- $MNS(i, j)$ ： $N(i, j)$ 的最大不相交子集，大小为 $Size(i, j)$ 。
- 当 $i = 1$ 时，

$$MNS(1, j) = N(1, j) \begin{cases} \emptyset & j < \pi(1) \\ \{(1, \pi(1))\} & j \geq \pi(1) \end{cases}$$

- 当 $i > 1$ 时，
 - 若 $j < \pi(i)$ ：

$$\begin{aligned} N(i, j) &= N(i-1, j) \\ Size(i, j) &= Size(i-1, j) \end{aligned}$$

- 若 $j \geq \pi(i)$ 且 $(i, \pi(i)) \in MNS(i, j)$ ：
 $\forall (t, \pi(t)) \in MNS(i, j)$ 有 $t < i$ 且 $\pi(t) < \pi(i)$ ，此时 $MNS(i, j) - \{(i, \pi(i))\}$ 是 $N(i-1, \pi(i)-1)$ 的最大不相交子集。
- 总结为：
 - 当 $i = 1$ 时，

$$Size(1, j) = \begin{cases} 0 & j < \pi(1) \\ 1 & j \geq \pi(1) \end{cases}$$

- 当 $i > 1$ 时，

$$Size(i, j) = \begin{cases} Size(i-1, j) & j < \pi(i) \\ \max\{Size(i-1, j), Size(i-1, \pi(i)-1) + 1\} & j \geq \pi(i) \end{cases}$$

- 解释： i 等于 1 只能根据是否出界判断 $size$ ， i 大于 1 时候如果出界就不选，如果没出界，可以自行选择是否选这条线（哪个大选哪个），不选就和上面情况一样，选了就是在 i 去掉 1、 j 变成当前这条线的终点的左边作为边界的情况上加 1。
- 实现：

```
// 最优值  
void MNS(int C[], int n, int **size) {  
    // 初始化
```

```

for (int j = 1; j < C[1]; j++) size[1][j] = 0;
for (int j = C[1]; j <= n) size[1][j] = 1;

for (int i = 2; i < n; i++) {
    for (int j = 1; j < C[i]; j++) size[i][j] = size[i-1][j];
    for (int j = C[i]; j <= n; j++) {
        // 不选 or 选
        size[i][j] = max(size[i-1][j], size[i-1][C[i]-1]+1);
    }
}

// 不可能出界，只需要n-1行的内容即可，不需要多余计算第n行的其他n-1列
size[n][n] = max(size[n-1][n], size[n-1][C[n]-1]+1)
}

// 最优解
void TraceBack(int C[], int **size, int n, int Net, int &m) {
    int j = n, m = 0;
    for (int i = n; i > 1; i--) {
        // 被选中的线
        if (size[i][j] != size[i-1][j]) {
            // 更新记录数组和边界
            Net[m++] = i;
            j = C[i] - 1;
        }
    }
}

// 若j依然包含第一根线，则直接选中
if (j >= C[1]) Net[m++] = 1;
}

```

- 复杂度：
 - 最优值：时空都是 $O(n^2)$ 。
 - 最优解：基于最优值，时间 $O(n)$ 。

流水作业调度

- 问题描述：见[回溯：批处理作业调度](#)。
- 解决方法：
 - π ：最优调度
 - $T(S, t)$ ：M2等时间t后才能用的情况下完成S中作业的最短用时。
 - $T(N, 0) = \min_{1 \leq i \leq n} \{a_i + T(N - \pi(1), b_i)\}$
 - $T(S, t) = \min_{i \in S} \{a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$
 - Johnson不等式：如果作业 i 、 j 满足 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$ ，则这两个作业满足Johnson不等式。

设 π 是作业集 S 在机器 M_2 的等待时间为 t 时的任一最优调度。若 $\pi(1)=i, \pi(2)=j$ 。则由动态规划递归式可得:

$$T(S, t) = a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\}) = a_i + a_j + T(S - \{i, j\}, t_{ij})$$

$$\begin{aligned} \text{其中, } t_{ij} &= b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} \\ &= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\} \\ &= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\} \\ &= b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\} \end{aligned}$$

- 交换上述 i, j , 则

$$\text{需的加工时间为 } T'(S, t) = a_i + a_j + T(S - \{i, j\}, t_{ji})$$

$$\text{其中 } t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_j, a_j\}$$

当作业 i 和 j 满足Johnson不等式时, 有

$$\begin{aligned} \max\{-b_i, -a_j\} &\leq \max\{-b_j, -a_i\} \\ a_i + a_j + \max\{-b_i, -a_j\} &\leq a_i + a_j + \max\{-b_j, -a_i\} \\ \max\{a_i + a_j - b_i, a_i\} &\leq \max\{a_i + a_j - b_j, a_j\} \\ \max\{t, a_i + a_j - b_i, a_i\} &\leq \max\{t, a_i + a_j - b_j, a_j\} \end{aligned}$$

- 因此对于该问题, 必然存在最优调度 π , 使得当 i, j 满足Johnson不等式。即调度满足Johnson 法则当且仅当对任意 $i < j$ 有 $\min\{b_{\pi(i)}, a_{\pi(j)}\} \geq \min\{b_{\pi(j)}, a_{\pi(i)}\}$, 因此所有满足Johnson 法则的调度均为最优调度。
- 实现: Johnson算法 (1上快的先做, 2上快的后做)
 1. 令 $N_1 = \{i | a_i < b_i\}$ 、 $N_2 = \{i | a_i \geq b_i\}$;
 2. N_1 中按照 a_i 非减序排列, N_2 按照 b_i 非增序排列;
 3. N_1 接 N_2 即为最优调度。
- 复杂度: 时间主要花在排序上, $O(n \cdot \log n)$, 空间 $O(n)$ 。

(重要) 0-1背包问题

- 问题描述: 给定 n 种物品和一背包。物品 i 的重量是 w_i , 其价值为 v_i , 背包的容量为 C 。问应如何选择装入背包的物品, 使得装入背包中物品的总价值最大?
- 解决思路:
 - 问题解读: 0-1背包问题是一个特殊的整数规划问题。

$$\max \sum_{i=1}^n v_i x_i$$

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\}, 1 \leq i \leq n \end{cases}$$

- 定义 $m(i, j)$ 为表示当背包剩余容量为 j , 且只能从第 i 个物品到第 n 个物品 (即可选物品集合为 $\{i, i+1, \dots, n\}$) 中进行选择时, 所能获得的最大价值。
- 递推公式:

$$m(i, j) = \begin{cases} \max\{m(i+1, j), & m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

- 递归出口：处理到最后一个物品时，只需要直接判断能否装入即可。

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

- 实现：

```
/**
 * 0-1 背包问题动态规划求解
 *
 * @param v 价值数组 (values), 下标从 1 到 n
 * @param w 重量数组 (weights), 下标从 1 到 n
 * @param c 背包总容量 (capacity)
 * @param n 物品数量
 * @param m 动态规划二维数组 (DP table), m[i][j] 表示从第 i 个物品到第 n 个物
品, 容量为 j 时的最大价值
 */
template<class Type>
void knapsack(Type v[], int w[], int c, int n, Type **m) {

    // --- 第一步: 处理最后一个物品 n (边界条件) ---
    // 计算物品 n 在容量允许范围内能否放入
    // jMax 是指当前物品 n 放不下的最大容量分界点
    int jMax = min(w[n] - 1, c);

    // 1. 容量 j 小于物品 n 的重量 w[n]: 装不下, 价值为 0
    for (int j = 0; j <= jMax; j++)
        m[n][j] = 0;

    // 2. 容量 j 大于等于 w[n]: 装得下, 价值为 v[n]
    for (int j = w[n]; j <= c; j++)
        m[n][j] = v[n];

    // --- 第二步: 逆序循环处理第 n-1 到第 2 个物品 ---
    // 我们只需要计算到 i=2, 因为 i=1 (最终结果) 只需要计算 m[1][c] 一个值, 不需
要算出整行
    for (int i = n - 1; i > 1; i--) {
        jMax = min(w[i] - 1, c);

        // 分段 1: 背包容量 j 不足 w[i], 肯定装不下物品 i
        // 策略: 只能继承“后面对物品 i+1...n 的选择结果”
        for (int j = 0; j <= jMax; j++)
            m[i][j] = m[i + 1][j];

        // 分段 2: 背包容量 j 足够装下物品 i
        // 策略: 决策“装”还是“不装”, 取最大值
        for (int j = w[i]; j <= c; j++)
```

```

        m[i][j] = max(m[i + 1][j], m[i + 1][j - w[i]] + v[i]);
    }

    // --- 第三步：单独计算最终结果 m[1][c] ---
    // 这一步是为了节省计算量，因为我们只需要知道容量为 c 时的最大价值
    m[1][c] = m[2][c]; // 先假设不装物品 1
    if (c >= w[1])      // 如果装得下，尝试装入并比较
        m[1][c] = max(m[1][c], m[2][c - w[1]] + v[1]);
}

```

- 复杂度：时间 $O(nc)$ 。

最优二叉搜索树

- 问题描述：
 - 二叉搜索树：
 - 左子树 < 根节点（如果子树非空）
 - 根节点 < 右子树（如果子树非空）
 - 左右子树都是二叉排序树
 - 随机情况下平均查找长度和 $\log n$ 等数量级
 - 期望耗费：k 为关键字（成功），d 为范围（失败）

二叉搜索树的期望耗费

- 搜索成功与不成功的概率

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$

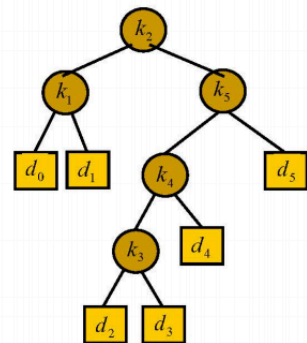
- 二搜索树的期望耗费

$E(\text{search cost in } T)$

$$= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}_T(d_i) + 1) \cdot q_i$$

$$= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i$$

- 有 n 个节点的二叉树的个数为：
- 穷举搜索法的时间复杂度为指数级 $\Omega(4^n / n^{3/2})$



- 解决方法：
 - i 和 j ：表示关键字的索引范围。我们在计算包含关键字 k_i 到 k_j 的这棵子树。
 - $m(i, j)$ ：表示包含关键字 $k_i \dots k_j$ 的最优二叉搜索树的期望搜索代价（总耗费）。这是我们需要求出的最终答案。
 - $w_{i,j}$ ：表示从 i 到 j 范围内所有节点（包括内部节点 p 和叶子节点 q ）的概率之和（权重和）。
 - k ：表示我们在 i 到 j 之间选定哪个节点作为根节点。

- 递推方程：

$$m(i, j) = w_{ij} + \min_{i \leq k \leq j} \{m(i, k-1) + m(k+1, j)\}$$

边界条件： $m(i, i-1) = 0$

- 增加的代价： $w_{i,j}$ 。当 k 变成根节点，左子树和右子树里的所有节点的深度都会比它们单独作为树时加 1（因为它们被挂在了 k 下面）。所有节点的深度加 1，意味着总期望代价增加了“所有节点的概率之和 $\times 1$ ”，也就是 $w_{i,j}$ 。
- 优化：用 $s[i][j]$ 记录范围是 (i, j) 时取得最优解的根节点位置。数学上可以证明，大区间 $[i, j]$ 的最优根节点 k ，一定落在两个子区间的最优根节点之间。

$$m(i, j) = w_{ij} + \min_{s[i][j-1] \leq k \leq s[i+1][j]} \{m(i, k-1) + m(k+1, j)\}$$

边界条件： $m(i, i-1) = 0$

- 实现：

```
/* `n`**：关键字（实节点）的数量。
`a[]`：失败概率（虚节点/叶子节点  $d_i$  的概率，通常记为  $q_i$ ）。
`b[]`：成功概率（实节点/内部节点  $k_i$  的概率，通常记为  $p_i$ ）。
`m[][]`：DP表，`m[i][j]` 存储从第  $i$  个到第  $j$  个关键字构成的最优树的期望代价。
`s[][]`：根节点表，`s[i][j]` 记录取得最优解时的根节点位置（用于后续构造树）。
`w[][]`：权重表，`w[i][j]` 存储区间  $[i, j]$  内所有  $a$  和  $b$  的概率之和。
*/
void OptimalBinarySearchTree(int a[], int b[], int n, int **m, int **s, int **w)
{
    // 初始化边界条件，空树，只有虚节点
    for (int i = 0; i <= n; i++) {
        w[i + 1][i] = a[i];
        m[i + 1][i] = 0;
    }

    // 区间长度r
    for (int r = 0; r < n; r++) {
        // 区间 i, j
        for (int i = 1; i <= n - r; i++) {
            int j = i + r;
            // 当前区间权重=原来的[i, j-1]加上新进入的关键字b[j]和新区间a[j]
            w[i][j] = w[i][j - 1] + a[j] + b[j];
            // 设置k为i
            m[i][j] = m[i + 1][j];
            s[i][j] = i;
            for (int k = i + 1; k <= j; k++) {
                int t = m[i][k - 1] + m[k + 1][j];
                if (t < m[i][j]) { // 不算根节点贡献，直接更新
                    m[i][j] = t;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

```
        }
    }
    m[i][j] += w[i][j]; // 根节点的贡献
}
}
```

- 复杂度： $O(n^3)$ 。

贪心

基础

概念

贪心算法总是作出**当前看来最好的选择**，不从整体最优考虑，仅选择**某种意义上的局部最优**；其目标是希望最终结果达成**整体最优**。可对部分问题产生整体最优解。

基本要素

- **最优子结构性质**
- **贪心选择性质**：所求问题的整体最优解可以通过一系列局部最优选择达到。

VS. 动态规划

- 只关注当前状态，而DP依赖于子问题的解（状态）
- 自顶向下 VS. 自底向上

一般理论

实例

活动安排问题

- 问题描述：在所给的活动集合中选出最大的相容活动子集合。
- 基础定义：假设有一个包含 n 个活动的集合 $E = \{1, 2, \dots, n\}$ ，这些活动都需要使用同一个公共资源，但在同一时间内，该资源只能供一个活动使用。每个活动 i 都有一个**开始时间** s_i 和一个**结束时间** f_i ，活动占用时间区间为半开区间 $[s_i, f_i)$ ，如果两个活动 i 和 j 的时间区间不重叠（即 $s_i \geq f_j$ 或 $s_j \geq f_i$ ），则称它们是**相容的**。
- 解决思路：首先已知所有活动按照**结束时间** f_i **非减序**排列（即 $f_1 \leq f_2 \leq \dots \leq f_n$ ）。总是选择**具有最早完成时间**且与当前已选活动相容的活动（前者由已排好序的数组保证），也就是若被检查的活动 i 的开始时间小于上一个选择的活动的结束时间（不相容）则不选择活动 i ，否则将 i 加入集合 A 中。
- 正确性：数学归纳法可证
- 实现：

```
// s[] 和 f[] 已经按照 f[] 非减序排好序
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1] = true; // 贪心选择：第一个活动总是被选入
    int j = 1;    // j 记录最近一次加入集合A的活动索引

    for (int i = 2; i <= n; i++) {
        // 核心判断：当前活动开始时间 >= 上一个选中活动的结束时间
        if (s[i] >= f[j]) {
            A[i] = true;
            j = i; // 更新 j 为当前选中的活动
        } else {
            A[i] = false;
        }
    }
}
```

- 复杂度：一次遍历 $O(n)$ ，若是乱序需要考虑排序所需的 $O(n \cdot \log n)$ 。

背包问题（可拆重量）

- 思想：将各种物品根据单位重量价值排序，先装单价高的。
- 实现：

```
void Knapsack(int n, float M, float v[], float w[], float x[])
{
    // 根据v、w计算单价并排序，重排v、w
    Sort(n, v, w);
    int i;
    for (i=1; i<=n; i++) x[i]=0;
    float c=M;
    for (i=1; i<=n; i++) {
        if (w[i]>c) break;
        x[i]=1;
        c-=w[i];
    }
    // 装不满就记下比例
    if (i<=n) x[i]=c/w[i];
}
```

- 复杂度：排序， $O(n \cdot \log n)$

最优装载问题

- 问题描述：有一批集装箱要装上一艘载重量为 c 的轮船。集装箱 i 的重量为 w_i ，轮船的总载重量为 c 。确定在装载体积不受限制的情况下，将**尽可能多**的集装箱装上轮船。这里的优化目标是集装箱的**数量**最大化，而不是总重量最大化。
- 解决思路：重量最轻者先装。
- 实现：

```

template<class Type>
void Loading(int x[], Type w[], Type c, int n)
{
    // x存储最终结果, w存每个重量, c为剩余载重量, n为集装箱总数
    // t存储排序后的集装箱编号
    int *t = new int [n+1];
    Sort(w, t, n);
    for (int i = 1; i <= n; i++) x[i] = 0;
    for (int i = 1; i <= n && w[t[i]] <= c; i++) {
        x[t[i]] = 1;
        c -= w[t[i]];
    }
}

```

- 复杂度：集中于排序， $O(n \cdot \log n)$ 。

哈夫曼编码

- 基本定义：
 - 前缀码：对每一个字符规定一个比特串作为代码，并要求任意字符的代码都不是其他字符代码的前缀，这种编码称为前缀码
 - 最优前缀码：使平均码长达到最小的前缀码编码方案称为给定编码字符集C的最优前缀码。
 - 平均码长定义为： $B(T) = \sum_{c \in C} f(c)d_T(c)$ (f 为频率, d 为 c 在树中的深度即码长)
 - 表示最优前缀码的二叉树总是一棵完全二叉树（树中任意结点都有两个儿子结点）
- 问题描述：根据字符出现频率进行字符编码，频率越高，码越短。
- 解决：自底向上构造最优前缀码的二叉树T，以 $|C|$ 个叶子结点（森林）开始，执行 $|C| - 1$ 次合并产生最终树。以f为键值的优先队列Q用在贪心选择时，有效的确定需要合并的两棵具有最小频率的树产生新的树，插入优先队列Q， $|C| - 1$ 次合并后只剩一棵树T。
- 实现：

```

MinHeap<Huffman<type>> Q(1); //建立优先队列
Q.Initialize(w, n, n);
//反复合并最小频率树
Huffman<Type> x, y;
for (int i=1; i<n; i++) {
    Q.DeleteMin(x);
    Q.DeleteMin(y);
    z.MakeTree(0, x.tree, y.tree);
    x.weight += y.weight;
    x.tree = z;
    Q.insert(x);
}

```

- 复杂度：初始化建堆需要 $O(n)$ ，每次执行中的删除和插入操作复杂度为 $O(\log n)$ ，共 $n - 1$ 次，总的来说复杂度为 $O(n \cdot \log n)$ 。

- 正确性证明：

- 贪心选择性质：假设 T 是一棵最优前缀码树（最优解），但它可能不是按照贪心算法生成的。设 x 和 y 是字符集中频率最小的两个字符。 b 和 c 是树 T 中深度最大的两个兄弟叶子节点（位于最底层）。显然 $f(x) \leq f(b)$ 且 $f(y) \leq f(c)$ 。在树 T 中，把 x 和 b 的位置交换，把 y 和 c 的位置交换，得到了一棵新树 T' 。此时在 T' 中，最小的 x, y 变成了最底层的兄弟。有公式：

$B(T) - B(T') = (f(b) - f(x))(d_T(b) - d_T(x)) + (f(c) - f(y))(d_T(c) - d_T(y)) \geq 0$ 。所以 $B(T') \leq B(T)$ 。因为 T 已经是最优的，所以 T' 的成本不可能比 T 更小，只能相等。这意味着 T' 也是一棵最优树，且它包含了我们的贪心选择。由此可得，我们总是可以从贪心选择出发找到最优解。

- 最优子结构性质（反证法）多一个 ' 就少一组 $f(x) + f(y)$ 确定相等关系（不去掉 x, y 只加上 z ），假设不是最优解，找到两组关系，通过比大小展示假定的最优前缀码是不成立的，从而证明结果：

设 T 表示字符集 C 的一个最优前缀码的完全二叉树。 C 中字符 c 的出现频率 $f(c)$ 。

设 x 和 y 是树 T 中的两个叶子且为兄弟， z 是它们的父亲。若将 z 看作具有频率 $f(z) = f(x) + f(y)$ 的字符，则树 $T' = T - \{x, y\}$ 表示字符集 $C' = C - \{x, y\} \cup \{z\}$ 的一个最优前缀码。

证明：（思路：反证法，以及建立 T' 与 T 关系）

1) 对任意 $c \in C - \{x, y\}$ ，有 $d_T(c) = d_{T'}(c)$ ，故 $f(c) d_T(c) = f(c) d_{T'}(c)$

2) 对 x, y 来说， $d_T(x) = d_T(y) = d_{T'}(z) + 1$ ，故

$$f(x) d_T(x) + f(y) d_T(y) = (f(x) + f(y))(d_{T'}(z) + 1) = f(x) + f(y) + f(z) d_{T'}(z)$$

综合1)、2) 知， $B(T) = B(T') + f(x) + f(y)$ 。

(反证法) 若 T' 表示的字符集 C' 的前缀码不是最优的，则有 T'' 表示的 C' 的前缀码，使得 $B(T'') < B(T')$ 。由于 z 被看作 C' 中的一个字符，故 z 在 T'' 中是一树叶。若将 x 和 y 加入树中作为 z 的儿子，则得到表示字符集 C 的前缀码的二叉树 T''' ，且有

$$B(T''') = B(T'') + f(x) + f(y) < B(T') + f(x) + f(y) = B(T)$$

这与 T 是最优性矛盾。故 T' 表示的 C' 的前缀码是最优的。

单源最短路径Dijkstra算法

- 问题描述：给定带权有向图 $G = (V, E)$ ，其中每条边的权是非负实数。另外，还给定 V 中的一个顶点，称为源。现在要计算从源到所有其它各顶点的最短路长度，这里路的长度是指路上各边权之和。
- 解决思路：设置顶点集合 S 并不断地作贪心选择来扩充这个集合。一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知。初始时， S 中仅含有源。设 u 是 G 的某一个顶点，把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的特殊路径，并用数组 $dist$ 记录当前每个顶点所对应的最短特殊路径长度。Dijkstra算法每次从 $V - S$ 中取出具有最短特殊路长度的顶点 u ，将 u 添加到 S 中，同时对数组 $dist$ 作必要的修改。一旦 S 包含了所有 V 中顶点， $dist$ 就记录了从源到所有其它顶点之间的最短路径长度。
- 实现：

```

dist[v]=0, s[v] =true;      //初始化
for (int i=1; i<n; i++) {    //遍历n-1次, 逐渐扩充S
    int temp = maxint, u = v;
    for (int j=1; j<=n; j++) // 找与S距离最近的节点u
        if ( (!s[j]) && (dist[j] < temp)) { u = j; temp = dist[j]; }
    s[u] =true;
    for(int j=1; j<=n; j++) { //根据新加入S的节点u, 更新与不在S中的节点j的距离
        if( (!s[j]) && (c[u][j] < maxint)) {
            Type newdist = dist[u] + c[u][j];
            if ( newdist < dist[j] )
                { dist[j]=newdist; prev[j]=u; } //prev[j]为节点j在路径上的前一个
节点
        }
    }
}
}

```

- 复杂性: $O(n^2)$ 。
- 正确性:
 - 贪心选择性质: 设从源到 u (不在 S 的所有节点) 的**最短路径长度** $dist[u]$ 。如果存在一条从源 v 到 u 且长度比 $dist[u]$ 更短的路, 设这条路初次走出 S 之外到达的顶点为 $x \in V - S$, 然后徘徊于 S 内外若干次, 最后离开 S 到达 u 。在这条路上, 分别记 $d(v, x)$, $d(x, u)$, $d(v, u)$ 为顶点 v 到顶点 x , 顶点 x 到顶点 u , 顶点 v 到顶点 u 的路长。 $dist[x] \leq d(v, x)$, $d(v, x) + d(x, u) = d(v, u) < dist[u]$ 。由于边权非负, 即 $d(x, u) \geq 0$, 从而**推得** $dist[x] < dist[u]$, **与假设矛盾**。
 - 最优子结构性质: 没有。

最小生成树Prim算法

- 问题描述:
 - 设 $G = (V, E)$ 是无向连通带权图, 即一个网络。 E 中每条边 (v, w) 的权为 $c[v][w]$ 。如果 G 的子图 G' 是一棵包含 G 的所有顶点的树, 则称 G' 为 G 的生成树。**生成树上各边权的总和称为该生成树的耗费**。在 G 的所有生成树中, 耗费最小的生成树称为 G 的**最小生成树**。网络的最小生成树在实际中有广泛应用。例如, 在设计通信网络时, 用图的顶点表示城市, 用边 (v, w) 的权 $c[v][w]$ 表示建立城市 v 和城市 w 之间的通信线路所需的费用, 则最小生成树就给出了建立通信网络的最经济的方案。
 - 最小生成树性质MST: 设 $G = (V, E)$ 是连通带权图, U 是 V 的真子集。如果 $(u, v) \in E$, 且 $u \in U$, $v \in V - U$, 且在所有这样的边中, (u, v) 的权 $c[u][v]$ 最小, 那么一定存在 G 的一棵最小生成树, 它以 (u, v) 为其中一条边。
- 解决方法: 构造 G 的最小生成树的Prim算法的基本思想是: 首先置 $S = \{1\}$, 然后, 只要 S 是 V 的真子集, 就作如下的**贪心选择** (在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树):
 1. 选取满足条件 $i \in S$, $j \in V - S$, 且 $c[i][j]$ 最小的边, **将顶点 j 添加到 S 中**。更新 S 中节点与其它节点距离。
 2. 这个过程一直进行到 $S = V$ 时为止。
- 实现:

```

for (int i = 1; i < n; i++) { //循环n-1次
    Type min = inf, j = 1;
    for(int k=2; k <= n; k++) { //选择加入S的节点
        if((lowcost[k] < min) && (!s[k])) { // s表示是否加入U
            min = lowcost[k];
            j = k;
        }
    }
    cout << j << ' ' << closet[j] << endl;
    s[j] = true;
    for (int k = 2; k <= n; k++) { // 更新S中的节点与其它节点距离
        if((c[j][k] < lowcost[k]) && (!s[k])) {
            lowcost[k] = c[j][k];
            closet[k] = j;
        }
    }
}
}

```

- 复杂度： $O(n^2)$ 。

最小生成树Kruskal算法

- 算法描述：Kruskal算法构造 G 的最小生成树的基本思想：首先将 G 的 n 个顶点看成 n 个孤立的连通分支。将**所有的边按权从小到大排序**。然后从第一条边开始，**依边权递增的顺序查看每一条边**，并按下述方法连接2个不同的连通分支：
 1. 当查看到第 k 条边 (v, w) 时，如果端点 v 和 w 分别是当前2个不同的连通分支 T_1 和 T_2 中的顶点时，**就用边 (v, w) 将 T_1 和 T_2 连接成一个连通分支，然后继续查看第 $k+1$ 条边**
 2. 如果端点 v 和 w 在当前的同一个连通分支中，就直接再查看第 $k+1$ 条边。这个过程一直进行到只剩下一个连通分支时为止。
- 实现：

```

// 循环条件：
// e：堆中还有剩余的边可供选择（e 代表剩余边的数量）
// k < n-1：已选中的边数还不够 n-1 条（生成树需要 n-1 条边）
while( e && k < n-1 ){
    EdgeNode<int> x; // 定义一个临时变量 x，用来存储当前要处理的边

    // 【贪心选择步骤】
    // 从最小堆 H 中取出权值最小的一条边，赋值给 x
    // 这一步体现了 Kruskal 的核心思想：总是优先看最小的边
    H.DeleteMin(x);

    e--; // 堆中剩余边的计数减 1

    // 【并查集查找 (Find)】
    // 查找边 x 的起点 u 属于哪个连通分量（集合代表元为 a）
    int a = U.Find(x.u);

```

```

// 查找边 x 的终点 v 属于哪个连通分量（集合代表元为 b）
int b = U.Find(x.v);

// 【判断是否构成环】
// 如果 a != b，说明顶点 u 和 v 分属不同的连通分量
// 连接它们不会在图中形成环
if( a != b ) {
    // 将这条边 x 加入到结果数组 t 中
    // k++ 表示已选边数加 1
    t[k++] = x;
    // 【并查集只是合并（Union）】
    // 将 u 所在的集合 a 和 v 所在的集合 b 合并成一个集合
    // 这代表这两个连通分量现在连通了
    U.Union(a, b);
}
// 如果 a == b，说明 u 和 v 已经在同一个连通分量里了，
// 再连这条边就会形成环，所以直接跳过（这里隐式地丢弃了这条边）
}

```

- 复杂度： $O(e \cdot \log e)$ ，处理边时遍历每条边进行从堆里取出的操作，单次复杂度为 $O(\log e)$ ，合并时复杂度为 $O(e)$ ，对比Prim算法， $e = (n^2)$ 则Kruskal好， $e = o(n^2)$ 则Prim好。

多机调度问题

- 问题描述：
 - **多机调度问题**要求给出一种作业调度方案，使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。
 - 约定，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。
 - 这个问题是**NP完全问题**，到目前为止还没有有效的解法。对于这一类问题，用贪心选择策略有时可以设计出较好的近似算法。
- 解决方法：采用**最长处理时间作业优先**的贪心选择策略可以设计出解多机调度问题的较好的近似算法。按此策略，当 $n \leq m$ 时，只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可。当 $n > m$ 时，首先将 n 个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。
- 复杂度： $O(n \cdot \log n)$ 。

分治 VS. 动态规划 VS. 贪心

类型	分治法	动态规划	贪心算法
子问题结构	子问题间独立	多个子问题重复	一个子问题
子问题数	多个	多个	一个
子问题独立	独立	子问题相关	独立
贪心选择性质	无	无	满足

类型	分治法	动态规划	贪心算法
最优子结构	无要求	必须满足	必须满足
问题与求解次序	自顶向下求解子问题（递归）	自底向上解子问题，再求更大规模问题	先贪心选择，再解决子问题

回溯法

基础

- 回溯法的深度优先搜索策略：
 - 有许多问题，当需要找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。回溯法的基本做法是搜索，或是一种组织得井井有条的能避免不必要搜索的穷举式搜索法。这种方法适用于解一些组合数相当大的问题。
 - 回溯法在问题的解空间树中，按深度优先策略，从根结点出发搜索解空间树。算法搜索至解空间树的任意一点时，先判断该结点是否包含问题的解。如果肯定不包含则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。
 - 回溯法求问题的所有解时，只要回溯到根，且根节点的所有子树都已被搜索到才结束。
- 问题的解空间：
 - 问题的解向量：回溯法希望一个问题的解能够表示成一个 n 元式 $(x_1, x_2, \dots, x_n)$ 的形式。
 - 显约束：对分量 x_i 的取值限定。
 - 隐约束：为满足问题的解而对不同分量之间施加的约束。
 - 解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。
 - 注意：同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小（存储量少，搜索方法简单）。
- 生成问题状态的基本方法
 - 扩展结点：一个正在产生儿子的结点称为扩展结点
 - 活结点：一个自身已生成但其儿子还没有全部生成的节点称做活结点
 - 死结点：一个所有儿子已经产生的结点称做死结点
 - 深度优先的问题状态生成法：如果对一个扩展结点 R ，一旦产生了它的一个儿子 C ，就把 C 当作新的扩展结点。在完成对子树 C （以 C 为根的子树）的穷尽搜索之后，将 R 重新变成扩展结点，继续生成 R 的下一个儿子（如果存在）
 - 宽度优先的问题状态生成法：在一个扩展结点变成死结点之前，它一直是扩展结点
 - 回溯法：为了避免生成那些不可能产生最佳解的问题状态，要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活结点，以减少问题的计算量。**具有限界函数的深度优先生成法称为回溯法**
- 基本思想：
 - 针对所给问题，定义问题的解空间；

- 确定易于搜索的解空间结构；
- 以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。
- 常用剪枝函数：
 - 用**约束函数**在扩展结点处剪去不满足约束的子树；
 - 用**限界函数**剪去得不到最优解的子树。
- 显著特征：搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到当前扩展结点的路径。如果解空间树中从根结点到叶结点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。
- 算法框架
 - 递归回溯：一般情况使用

```
void backtrack(int t) {
    // 层级t大于最大深度n，直接输出当前解向量
    if (t > n) output(x);
    else {
        // 遍历当前层级所有可能的状态
        for (int i = f(n, t); i <= g(n, t); i++) {
            // 将当前状态确定为解向量的t分量
            x[t] = h(i);
            // 满足约束和限界则直接进行下一层级
            if (constraint(t) && bound(t)) backtrack(t + 1);
        }
    }
}
```

- 迭代回溯：采用树的非递归深度优先遍历算法

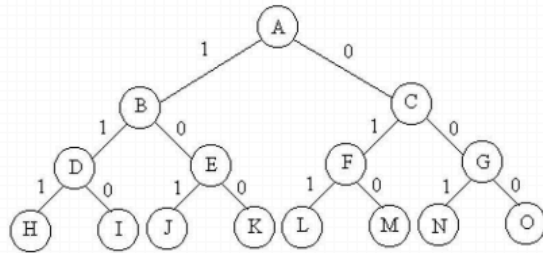
```
void iterativeBacktrack() {
    int t = 1; // t 表示当前递归的深度
    while (t > 0) {
        // f(n,t) 和 g(n,t) 定义了当前层 t 的搜索范围（下界和上界）
        // 这一步通常隐含了“寻找当前层下一个未尝试过的节点”的状态记忆
        if (f(n, t) <= g(n, t)) {
            // 遍历当前层 t 的所有可能取值 i
            for (int i = f(n, t); i <= g(n, t); i++) {
                x[t] = h(i);
                // 剪枝判断：检查显约束(constraint)和限界函数(bound)
                if (constraint(t) && bound(t)) {
                    // 3. 如果当前是完整解（到达叶子节点或满足特定条件）
                    if (solution(t)) {
                        output(x); // 输出解
                    }
                    // 4. 否则，如果还是部分解，则深度加深，继续搜索下一层
                } else {
                    t++;
                }
            }
        }
    }
}
```

```

    }
}
// 回溯：如果当前层没有可行解，或者已经遍历完所有可能
else {
    t--;
}
}
}

```

- 子集树算法框架：问题是从 n 个元素中找出满足某种性质的子集时，二叉树



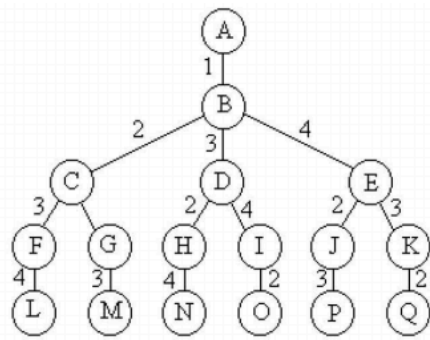
遍历子集树需 $O(2^n)$ 计算时间

```

void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=0;i<=1;i++) {
            x[t]=i;
            if (legal(t)) backtrack(t+1);
        }
}

```

- 排列树算法框架：问题是确定 n 个元素满足某种性质的排列，多叉树，使用了全排列的思想，只能解决规模很小的 n



遍历排列树需要 $O(n!)$ 计算时间

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=t;i<=n;i++) {
            swap(x[t], x[i]);
            if (legal(t)) backtrack(t+1);
            swap(x[t], x[i]);
        }
}
```

实例

装载问题：子集树

- 问题描述：有一批共 n 个集装箱要装上2艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且所有集装箱的总重量不超过两艘船的总载重（即 $\sum_{i=1}^n w_i \leq c_1 + c_2$ ）。确定是否有一个合理的装载方案可将这个集装箱装上这2艘轮船。如果有，找出一种装载方案。
- 解决思路：
 - 首先将第一艘轮船尽可能装满，将剩余的集装箱装上第二艘轮船，等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近该轮船可承载重量 c_1 。
 - 由此可知，装载问题等价于以下特殊的0-1背包问题：目标函数为 $\max \sum_{i=1}^n w_i x_i$ ，约束条件为 $\sum_{i=1}^n w_i x_i \leq c_1$ 。
 - 由于每个集装箱只有“装入第一艘船 ($x_i = 1$)”和“不装入第一艘船 ($x_i = 0$)”两种状态，因此问题的解空间是子集树
 - 剪枝：
 - 可行性约束函数（用于左子树）：判断选择当前集装箱后，是否超过第一艘船的载重量 c_1 。即 $\sum w_i x_i \leq c_1$ 。
 - 上界函数（用于右子树）：判断如果不选择当前集装箱，剩余的集装箱即使全装进去，总重量是否能超过当前已知的最优载重量 ($bestw$)。如果不能，则剪枝。
- 实现：

```
// 当前载重量cw、剩余集装箱重量r、当前最优载重量bestw、最优解向量bestx
void backtrack(int i) {
    // 叶子结点
```

```

    if (i > n) {
        if (cw > bestw) {
            bestw = cw;
            // 保存当前的解向量 x 到 bestx
            for (int j = 1; j <= n; j++) bestx[j] = x[j];
        }
        return;
    }

    // 去掉当前节点
    r -= w[i];

    // 左子树，放入
    if (w[i] <= c) {
        x[i] = 1;
        cw += w[i];
        backtrack(i + 1);
        // 恢复状态
        cw -= w[i];
    }

    // 右子树，不放入
    if (bestw < cw + r) {
        x[i] = 0;
        backtrack(i + 1);
    }

    // 恢复状态
    r += w[i];
}

```

- 复杂度： $O(2^n)$ ，某些情况优于动态规划算法。

批处理作业调度：排列树

- 问题描述：
 - 给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ ，每个作业必须先由机器1处理，然后由机器2处理，作业 i 需要机器 j 的处理时间为 t_{ji} 。
 - 对于一个确定的作业调度（即作业的加工顺序），设 F_{ji} 是作业 i 在机器 j 上完成处理的时间。
 - 所有作业在机器2上完成处理的时间之和称为该作业调度的完成时间和。批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案，使其完成时间和达到最小。即求一个排列 x ，使得 $\sum_{i=1}^n F_{x[i],2}$ 最小。
- 解决思路：
 - 需要确定 n 个作业的最佳加工顺序，这属于排列问题。因此，解空间是**排列树**。树的深度为 n ，每一层代表作业序列中的第 i 个位置放置哪个作业。
 - 时间计算逻辑：为了计算完成时间，需要跟踪两个状态变量：

1. f_1 : 机器1完成当前作业的时间。因为机器1是连续工作的, 所以它是所有已安排作业在机器1上的处理时间之和: $f_1 = f_1 + t_{x[j],1}$

2. $f_2[i]$: 作业序列中第 i 个作业在机器2上的完成时间。机器2开始处理第 i 个作业必须满足两个条件: 机器1处理完该作业, 且机器2处理完上一个作业:

$$f_2[i] = \max(f_2[i-1], f_1) + t_{x[j],2}$$

- 剪枝策略 (限界函数): 需要维护当前部分解的完成时间和 $f = f + f_2[i]$ 。如果在搜索过程中, 当前的累计完成时间和 f 已经大于或等于当前已知的最优解 $bestf$, 则说明该路径不可能产生更好的解, 直接剪去该子树 (即停止向下搜索)。

- 回溯: $+= \rightarrow -=$ 。

- 实现:

```
/*
int **M: 各作业所需的处理时间
int *x: 当前作业调度 (排列)
int *bestx: 最优排列
int *f2: 机器2完成处理时间, = max{当前任务机器1完成时间, 已经确定的上个位置i-1的机器2结束时间} + 机器2当前作业处理时间M[x[j]][2]
int f1: 机器1完成处理时间, = 上一个f1 + 当前任务在机器1上的处理时间M[x[j]][1]
int f: 完成时间和, = 上一个f + 当前任务放到位置i在机器2的完成时间f2[i]
int bestf: 最好完成时间和
int n: 作业数
*/
void Backtrack(int i) {
    // 【1. 递归终止条件】
    // 当 i > n 时, 说明所有作业都已安排完毕, 到达叶子节点
    if (i > n) {
        // 更新最优解
        for (int j = 1; j <= n; j++) bestx[j] = x[j];
        bestf = f;
    }
    else {
        // 【2. 遍历当前层 i 的所有选择】
        // 从 i 到 n 遍历, 尝试将每个剩余作业放到位置 i
        for (int j = i; j <= n; j++) {
            // 机器1是连续的, 直接累加当前作业 x[j] 的时间
            f1 += M[x[j]][1];

            // 计算当前作业 x[j] 在机器2上的完成时间
            // 注意: 如果是第1个作业(i=1), f2[0]应视为0
            f2[i] = max(f2[i-1], f1) + M[x[j]][2];

            // 累加总完成时间和
            f += f2[i];

            // 限界函数: 只有当前累计时间 f 小于最优解 bestf 时才继续
            if (f < bestf) {
                Swap(x[i], x[j]); // 交换, 将作业 x[j] 固定在位置 i
                Backtrack(i + 1); // 递归进入下一层
            }
        }
    }
}
```

```

        Swap(x[i], x[j]); // 回溯：恢复交换
    }

    // 回溯：状态恢复
    // 恢复 f 和 f1 的值，以便循环尝试下一个作业
    f -= f2[i];
    f1 -= M[x[j]][1];
}
}
}

```

- 复杂度： $O(n!)$ 。

符号三角形问题：子集树

- 问题描述：在一般情况下，符号三角形的第一行有 n 个符号。要求对于给定的 n ，计算有多少个不同的符号三角形，使其所含的“+”和“-”的个数相同。比如下图中+/-都是14个：

```

+  +  -  +  -  +  +
  +  -  -  -  -  +
    -  +  +  +  -
      -  +  +  -
        -  +  -
          -  -
            +

```

- 解决思路：
 - 生成规则总结为：
 - 两个同号下面都是“+”(即 $++ \rightarrow +$, $-- \rightarrow +$)。
 - 两个异号下面都是“-”(即 $+- \rightarrow -$, $-+ \rightarrow -$)。
 - 由于第一行元素确定后则三角形确定，所以只需要遍历第一行 n 个元素，每个元素2种可能，因此是一棵子集树。
 - 剪枝条件：
 - 一种符号的数量超过总共符号数量的一半即为错误解 ($count > half$)
 - 每次的流程为先确定第一行第 t 个元素，然后进行扩展（确定下面的可确定元素），然后判断是否符合剪枝条件，一直到 $t > n$ ，此时代表一种合适的解，解的数量+1
 - 用0代表“+”、1代表“-”，就可以利用异或操作推导下面 t 行的符号，并只需要使用一个变量记录符号数。
- 实现：

```

/*
t: 当前正在决定第一行的第 t 个符号
count: -的个数
(t * (t - 1) / 2 - count: +的个数

```

```

sum: 解的个数
p: 三角形数组, 第j行的前n-j+1个元素有意义
*/
void Backtrack(int t) {
    // 【1. 剪枝判断】
    // 如果当前 "+" (0) 的数量或 "-" (1) 的数量已经超过总数的一半, 剪枝
    if ((count > half) || (t * (t - 1) / 2 - count > half))
        return;

    // 【2. 递归终止条件】
    // 到达叶子节点 (第一行 n 个符号都定好了, 且没被剪枝, 说明找到一个解)
    if (t > n) {
        sum++;
    }
    else {
        // 【3. 遍历选择】
        // 第一行第 t 个位置可以是 0 或 1
        for (int i = 0; i < 2; i++) {
            p[1][t] = i; // 设定第一行第 t 个符号
            count += i; // 更新统计计数

            // 推导下方符号, 每多确定一个第一行的元素, 后续的t行就会对应产生一个元素
            for (int j = 2; j <= t; j++) {
                // 生成规则: 异或运算 (相同为0, 不同为1)
                // p[j][t-j+1] 是第 j 行的对应元素
                p[j][t-j+1] = p[j-1][t-j+1] ^ p[j-1][t-j+2];
                count += p[j][t-j+1]; // 统计该对角线上产生的新符号
            }

            // 【5. 递归进入下一层】
            Backtrack(t + 1);

            // 【6. 回溯: 恢复现场】
            // 将刚才那个循环里加上的 count 减回去
            for (int j = 2; j <= t; j++) {
                count -= p[j][t-j+1];
            }
            count -= i;
        }
    }
}

```

- 时间复杂度: 可行性计算需要 $O(n)$, 所以总的方法复杂度为 $O(n \cdot 2^n)$ 。

(重要) n后问题: 排列树

- 问题描述: 在 $n \times n$ 的棋盘上放置彼此不受攻击的n个皇后。等价于在 $n \times n$ 的棋盘上放置n个皇后, 任意两个皇后不放在同一行或同一列或同一斜线上。

- 解决思路：排列树
 - 解向量 $(x_1, x_2, \dots, x_n)$ ：第 i 行第 x_i 列
 - 显约束： $x \in [1, n]$ 。
 - 隐约束：不同列 ($x_i \neq x_j$)、不同一斜线 ($|i - j| \neq |x_i - x_j|$)
- 实现：

```
/**
 * 检查第k个皇后放置是否合法（是否与前k-1个皇后冲突）
 * @param k 当前要放置的皇后编号（第k行）
 * @return true-放置合法，false-放置冲突
 */
bool Queen::Place(int k)
{
    // 遍历已放置的前k-1个皇后
    for (int j = 1; j < k; j++)
        // 检查两种冲突情况：
        // 1. abs(k-j) == abs(x[j]-x[k]): 是否在同一对角线上
        // 2. x[j] == x[k]: 是否在同一列上
        if ((abs(k-j) == abs(x[j]-x[k])) || (x[j] == x[k]))
            return false; // 发生冲突，放置非法

    return true; // 无冲突，放置合法
}

/**
 * 回溯法求解N皇后问题
 * @param t 当前处理的行号（第t个皇后）
 */
void Queen::Backtrack(int t)
{
    if (t > n) // 递归出口：所有n个皇后都已成功放置
        sum++; // 找到一个可行解，解的数量+1
    else
        // 尝试在第t行的每一列放置皇后
        for (int i = 1; i <= n; i++) {
            x[t] = i; // 将第t个皇后放在第i列
            if (Place(t)) // 如果放置合法（不冲突）
                Backtrack(t+1); // 递归放置下一个皇后
            // 如果不合法，自动尝试下一列（隐式回溯）
        }
}
```

- 复杂度： $O(n * n!)$ 。

0-1背包问题：子集树

- 问题描述：略
- 解决思路：子集树

- 约束: $\sum_{i=1}^n w_i x_i \leq c_1$

- 实现:

```
/**
 * 0-1背包问题的回溯算法
 * @tparam Typew 重量的数据类型
 * @tparam Typep 价值的数据类型
 * @param i 当前考虑的物品编号（从1开始）
 *
 * 成员变量说明:
 *   n      - 物品总数
 *   c      - 背包容量
 *   w[]    - 各物品重量数组
 *   p[]    - 各物品价值数组
 *   cw     - 当前背包重量 (current weight)
 *   cp     - 当前背包价值 (current profit)
 *   bestp  - 当前最优价值 (best profit)
 */
template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Bound(int i) {

    // 计算背包剩余容量
    Typew cleft = c - cw;          // cleft = capacity left

    // 以当前已获得的价值为基础
    Typep b = cp;                  // b = bound (上界)

    //===== 贪心装入: 将能完整装入的物品依次装入 =====
    while (i <= n && w[i] <= cleft) {
        cleft -= w[i];             // 剩余容量减少
        b += p[i];                 // 价值增加
        i++;                       // 考虑下一个物品
    }

    //===== 部分装入: 将下一个物品部分装入 (分数背包) =====
    // 这是获得"上界"的关键: 允许物品拆分
    if (i <= n)
        b += p[i] / w[i] * cleft; // 按比例装入剩余容量

    // 返回价值上界
    return b;
}

template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Backtrack(int i) {

    //===== 递归出口 =====
    if (i > n) {                   // 已考虑完所有n个物品
        bestp = cp;               // 更新最优解
    }
}
```

```

        return;
    }

    //===== 左子树：选择放入第i个物品 =====
    if (cw + w[i] <= c) {          // 约束条件：放入后不超过背包容量
        cw += w[i];                // 更新当前重量（放入物品i）
        cp += p[i];                // 更新当前价值
        Backtrack(i + 1);          // 递归处理下一个物品
        cw -= w[i];                // 回溯：恢复重量（取出物品i）
        cp -= p[i];                // 回溯：恢复价值
    }

    //===== 右子树：选择不放入第i个物品 =====
    if (Bound(i + 1) > bestp)      // 剪枝条件：上界是否有可能超过当前最优解
        Backtrack(i + 1);        // 有希望则继续搜索
    // 否则剪枝，不再搜索这个分支
}

```

- 复杂度： $O(n \cdot 2^n)$ 。

最大团问题：子集树

- 问题描述：
 - 给定无向图 $G = (V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。
 - G 的完全子图 U 是 G 的团当且仅当 U 不包含在 G 的更大的完全子图中。 G 的最大团是指 G 中所含顶点数最多的团。
 - 如果 $U \subseteq V$ 且对任意 $u, v \in U$ 有 $(u, v) \notin E$ ，则称 U 是 G 的空子图。 G 的空子图 U 是 G 的独立集当且仅当 U 不包含在 G 的更大的空子图中。
 - G 的最大独立集是 G 中所含顶点数最多的独立集。
 - 对于任一无向图 $G = (V, E)$ 其补图 $\bar{G} = (V_1, E_1)$ 定义为： $V_1 = V$ ，且 $(u, v) \in E_1$ 当且仅当 $(u, v) \notin E$ 。
 - U 是 G 的最大团当且仅当 U 是 $\bar{G}$ 的最大独立集。
- 解决方法：子集树
 - 我们要决定图中的 n 个顶点，每一个是“选入团中”还是“不选入”。这构成了一棵二叉树，左分支代表“选入”，右分支代表“不选入”。
 - 剪枝策略
 - 左子树（选入）的约束条件（可行性剪枝）：如果要将顶点 i 加入当前团，必须保证 i 与当前团中已选的所有顶点都有边相连。如果有一个不连，就不能走左分支。
 - 右子树（不选入）的限界条件（最优化剪枝）：假设当前团里已有 cn 个顶点，还剩下 $n-i$ 个顶点待处理。如果 $cn + (n-i) \leq \text{bestn}$ （当前已有 + 剩余所有 $\leq$ 目前找到的最好结果），那么右分支就不必走了，因为走下去也不可能产生比当前最优解更好的结果。
- 实现：

```

void Clique::Backtrack(int i) {
    // 1. 到达叶子节点（递归出口）
    if (i > n) {
        // 更新最优解
        for (int j = 1; j <= n; j++) bestx[j] = x[j];
        bestn = cn; // cn 是当前团的大小 (current number)
        return;
    }

    // 2. 尝试进入左子树（放入顶点 i）
    int OK = 1;
    // 检查顶点 i 是否与已选入的顶点(x[j]==1)都相连
    for (int j = 1; j < i; j++) {
        if (x[j] == 1 && !a[i][j]) { // a[i][j]是邻接矩阵，0表示无边
            OK = 0;
            break;
        }
    }

    if (OK) { // 如果满足完全连接条件
        x[i] = 1;
        cn++; // 当前团大小 +1
        Backtrack(i+1);
        x[i] = 0; // 回溯：恢复现场
        cn--;
    }

    // 3. 尝试进入右子树（不放入顶点 i）
    // 限界函数：当前点数 + 剩余点数 > 当前最优解，才有必要继续
    if (cn + n - i > bestn) {
        x[i] = 0;
        Backtrack(i+1);
    }
}

```

- 复杂度： $O(n \cdot 2^n)$ 。

图的m着色问题：排列树

- 问题描述：给定无向连通图 G 和 m 种不同的颜色。用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色。是否有一种着色法使 G 中每条边的2个顶点着不同颜色。这个问题是图的 m 可着色判定问题。若一个图最少需要 m 种颜色才能使图中每条边连接的2个顶点着不同颜色，则称这个数 m 为该图的色数。求一个图的色数 m 的问题称为图的 m 可着色优化问题。
- 解决方法：
 - 解向量表示顶点 i 所着颜色 $x[i]$
 - 约束：相邻顶点颜色不重复
- 实现：

```

bool Color::Ok(int k)
{
    // 检查颜色可用性
    for (int j=1; j<=n; j++)
        // 如果顶点k与j相邻(a[k][j]==1) 且 颜色相同(x[j]==x[k]), 则返回false
        if ((a[k][j]==1) && (x[j]==x[k])) return false;
    return true;
}

void Color::Backtrack(int t)
{
    if (t > n)
    {
        sum++;
        bestx = x;
    }
    else
    {
        for (int i=1; i<=m; i++) // 尝试给第t个顶点着颜色 1 到 m
        {
            x[t] = i;           // 赋值颜色
            if (Ok(t))           // 检查颜色是否冲突
                Backtrack(t+1); // 如果不冲突, 递归处理下一个顶点
        }
    }
}

```

- 复杂度: $\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1)/(m - 1) = O(nm^n)$ 。

(重要) 旅行售货员问题: 排列树

- 问题描述: 设有p个城镇, 已知两两之间的距离, 一个售货员从某一城镇出发巡回售货, 如何选择路线, 能使得每个城镇经过且仅一次, 最后回到出发点, 总行程最短。
- 解决方法: 排列树
- 实现:

```

template<class Type>
void Traveling<Type>::Backtrack(int i)
{
    // === 1. 到达叶子节点 (处理找到的一条完整路径) ===
    if (i == n) {
        // 检查:
        // 1. 倒数第二个点到最后一个点是否有边? (a[x[n-1]][x[n]] != NoEdge)
        // 2. 最后一个点能否回到起点 1? (a[x[n]][1] != NoEdge)
        // 3. 总成本是否比当前最优成本更低? (或者当前还没有最优成本)
        if (a[x[n-1]][x[n]] != NoEdge && a[x[n]][1] != NoEdge &&
            (cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc || bestc ==
            NoEdge))
        {
            // 更新最优解

```

```

        for (int j = 1; j <= n; j++)
            bestx[j] = x[j]; // 记录最优路径

        // 更新最优成本：当前花费 + 最后一段进城 + 回起点的路
        bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1];
    }
}

// === 2. 递归扩展子节点（搜索中间过程） ===
else {
    for (int j = i; j <= n; j++)
    {
        // === 剪枝条件（Pruning） ===
        // 1. 上一个点 x[i-1] 到当前尝试的点 x[j] 是否有路？
        // 2. （当前花费 cc + 新路段花费）是否小于当前最优解 bestc？
        //    如果已经比最优解贵了，就没必要继续往下走了。
        if (a[x[i-1]][x[j]] != NoEdge &&
            (cc + a[x[i-1]][x[j]] < bestc || bestc == NoEdge))
        {
            // 交换位置，将 x[j] 放到当前位置 x[i]
            Swap(x[i], x[j]);
            // 累加花费
            cc += a[x[i-1]][x[i]];
            // 递归进入下一层
            Backtrack(i + 1);

            // === 回溯（Backtracking） ===
            // 恢复花费
            cc -= a[x[i-1]][x[i]];
            // 恢复位置交换
            Swap(x[i], x[j]);
        }
    }
}
}
}

```

- 复杂度：每次最坏情况更新需要 $O((n-1)!)$ ，每次更新best需要 $O(n)$ ，从而整个算法复杂度为 $O(n!)$ 。

圆排列问题：排列树

- 问题描述：给n个大小不等的圆 c_i ，排进一个矩形框内，要求各圆与矩形框底边相切，求从n个圆的所有排列中找出最小长度的圆排列。
- 解决方法：排列树。
- 实现：

```

void Circle::Compute()
{
    float low = 0, high = 0;
    for (int i = 1; i <= n; i++) {

```

```

        // 找出最左边的边界（圆心坐标 - 半径）
        if (x[i] - r[i] < low) low = x[i] - r[i];
        // 找出最右边的边界（圆心坐标 + 半径）
        if (x[i] + r[i] > high) high = x[i] + r[i];
    }
    // 如果当前排列的长度比记录的最小值 min 还小，则更新 min
    if (high - low < min) min = high - low;
}

float Circle::Center(int t)
{
    float temp = 0;
    // 遍历前面所有已放置的圆（从1到t-1）
    for (int j = 1; j < t; j++) {
        // 计算如果第t个圆与第j个圆相切，第t个圆的圆心坐标应该是多少
        float valuex = x[j] + 2.0 * sqrt(r[j] * r[t]);

        // 取最大值：因为第t个圆不能与前面任何一个圆重叠，
        // 所以它必须位于所有“潜在相切位置”的最右边。
        if (valuex > temp) temp = valuex;
    }
    return temp;
}

void Circle::Backtrack(int t)
{
    // 如果 t > n，说明所有圆都已经排好序了，到达叶子节点
    if (t > n)
        Compute(); // 计算当前排列的总长度，并更新最小值
    else {
        for (int j = t; j <= n; j++) {
            Swap(r[t], r[j]); // 【关键】交换半径数组，尝试将第j个圆放到位置t

            float centerx = Center(t); // 计算当前第t个圆的圆心横坐标

            // 下面这两行图片中被遮挡，但根据回溯法逻辑通常包含剪枝
            // 剪枝条件：如果当前计算出的部分长度已经超过了已知的最小长度，就没必要继续了

            if (centerx + r[t] + r[1] < min) {
                x[t] = centerx; // 记录第t个圆的圆心坐标
                Backtrack(t + 1); // 递归处理下一个位置
            }

            Swap(r[t], r[j]); // 【回溯】恢复现场，以便进行下一次循环尝试
        }
    }
}

```

- 复杂度： $O((n+1)!)$ 。

连续邮资问题

- 问题描述：假设国家发行了n种不同面值的邮票，规定每张信封最多贴m张邮票。要求对给定的n和m，给出邮票面值的最佳设计，在一张信封上可以贴出从邮资1开始增量为1的最大连续邮资区间。
- 解决方法：
 - 解向量： $x[1:n]$ ，表示不同邮票面值，从小到大， $x[1]=1$
 - 可行性约束：已选定 $x[1:i-1]$ ，最大连续邮资区间是 $[1:r]$ ，接下来 $x[i]$ 可取值范围是 $[x[i-1]+1 : r+1]$ 。
 - 确定 r 的值：引入 $y_i(j)$ ：用至多 m 张面值1到 x_i 的邮票贴 j 邮资时的最少邮票数：

$$y_i(j) = \min_{1 \leq t \leq m} \{t + y_{i-1}(j - tx_i)\}$$

$$y_1(j) = j$$

$$r_i = \max\{j | y_i(j) \leq m, y_i(j+1) > m\}$$

- 实现

```
void Stamp::Backtrack(int i, int r) // 第i张邮票序号，r-1为最大连续邮资
{
    // 搜索确定第i-1张邮票的面值x[i-1]后，更新贴出各个连续邮资值所需最少邮票数，与x[i-2]相关
    for(int j=0; j <= x[i-2]*(m-1); j++) //m张邮票，包含1张x[i-1]时，连续邮资范围[0, x[i-2]*(m-1)]
        if( y[j] < m )
            for(int k = 1; k <= m-y[j]; k++)
                if( y[j] + k < y[ j + x[i-1]*k ] )
                    y[ j + x[i-1]*k ] = y[j] + k;

    while( y[r] < maxint ) r++; // 求解第i个节点的最大连续邮资r

    if( i > n ) { // 到达叶节点，先深搜索完毕
        if( r-1 > maxvalue ) { //maxvalue 当前最大连续邮资
            maxvalue = r-1;
            for(int j=1; j <= n; j++) //x[]为获得最大连续邮资的第j张邮票面额
                bestx[j] = x[j];
        }
        return;
    }

    int *z = new int[maxl+1];
    for(int k = 1; k <= maxl; k++) z[k] = y[k];
    for(int j = x[i-1]+1; j <= r; j++) { // 搜索x[i], x[i]取值范围[ x[i-1]+1: r ]
        x[i] = j;
        Backtrack(i+1, r); //搜索第i+1个邮票面值
        for(int k = 1; k <= maxl; k++)
            y[k] = z[k];
    }
    delete[] z;
}
```

- 复杂度：指数级，只适用于n很小的情况。